Contents

I Numerical Analysis **3**

1 Rootfinding 4

2 Factorizations 5

3 Eigenvalue Computation 6

4 Exploiting Sparsity 7

5 Interpolation and Quadrature 8

II Nonconvex Optimization **9**

6 Unconstrained Optimization 9

7 Line-Search Methods 10

8 Trust-Region Methods 11

9 Optimizers for Deep Learning 12
9.1 Gradient Descent 12
9.2 Momentum 12

III Convex Optimization **14**

10 Linear Programming 15

11 Convex Programming 16

12 Semidefinite Optimization 17

13 Online Convex Optimization 18
13.1 Prediction from Expert Advice 18

14 Dynamical Systems 19

IV Reinforcement Learning **20**

15 Optimal Control 20
15.1 Bellman Equations 20

16 Value Learning 21
16.1 TD-Learning 21
16.2 SARSA and Q-Learning 21

17 Policy Optimization . . . 24
17.1 REINFORCE . . . 26
17.2 Natural Policy Gradients . . . 27
17.3 Conservative Policy Iteration . . . 29
17.4 Trust-Region Policy Optimization . . . 30
17.5 Proximal Policy Optimization . . . 32
17.6 Group-Relative Policy Optimization . . . 34

18 Actor-Critic Methods . . . 35
18.1 Determinstic Policy Gradients . . . 35
18.2 Soft Actor-Critic . . . 37

19 Goal-Conditioning . . . 38
19.1 Universal Value Function Approximators . . . 38
19.2 Hindsight Experience Replay . . . 38
19.3 Contrastive Reinforcement Learning . . . 39

20 Unsupervised Skill Discovery . . . 40
20.1 Noise-Contrastive Estimation . . . 40
20.2 Contrastive Predictive Coding . . . 40

21 Contrastive Reinforcement Learning . . . 42
21.1 Contrastive Learning . . . 42
21.2 CURL: Contrastive Representation Learning for RL . . . 43
21.3 Mutual Information as an Intrinsic Reward . . . 43
21.4 Diversity is All You Need (DIAYN) . . . 44
21.5 Mutual Information Skill Learning (MISL) . . . 45
21.6 Contrastive Intrinsic Control (CIC) . . . 46
21.7 Contrastive RL as Goal-Conditioned RL . . . 47
21.8 Goal-Conditioned Reinforcement Learning . . . 49
21.8.1 The Sparse Reward Problem and Hindsight Experience Replay . . . 50
21.8.2 Goal Representations . . . 52
21.8.3 Sub-Goal Generation and Curriculum Learning . . . 53
21.8.4 Synthesis: GCRL as a Unifying Lens . . . 54
21.9 Contrastive Representations for Temporal Reasoning . . . 54
21.10 Summary and Connections . . . 55

Part I

Numerical Analysis

1 Rootfinding

2 Factorizations

3 Eigenvalue Computation

4 Exploiting Sparsity

5 Interpolation and Quadrature

Part II

Nonconvex Optimization

6 Unconstrained Optimization

First-Order Necessary Conditions

If x^* is a local minimizer and $f \in C^1$ locally, then $\nabla f(x^*) = 0$; in other words, any local minimizer is a *stationary point*.

Second-Order Necessary Conditions

If x^* is a local minimizer and $f \in C^2$ locally, then $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*) \succcurlyeq 0$.

Second-Order Sufficient Conditions

If $f \in C^2$ locally, $\nabla f(x^*) = 0$, and $\nabla^2 f(x^*) \succ 0$, then x^* is a *strict* local minimizer.

Remark. The proofs of all three preceeding conditions are direct by Taylor expansion. ■

7 Line-Search Methods

Line-Search Methods

Each iteration of a line search method computes a search direction p_k and then updates via $x_{k+1} = x_k + \alpha_k p_k$, where $\alpha_k > 0$ is the *step size*.

- Most algorithms require p_k to be a descent direction ($p_k^\top \nabla f(x_k) < 0$) for convergence.
- The search direction usually has the form $p_k = -B_k^{-1} \nabla f(x_k)$ for an appropriate $B_k \succ 0$ symmetric and nonsingular.

For example, $B_k = I$ in gradient descent while $B_k = \nabla^2 f(x_k)$ in Newton's method.

TODO: Nocedal and Wright, *Numerical Optimization*.

8 Trust-Region Methods

Trust-Region Methods

9 Optimizers for Deep Learning

9.1 Gradient Descent

Minibatch Stochastic Gradient Descent

For $f : \mathbb{R}^d \to \mathbb{R}$ differentiable, stochastic gradient descent updates $w_k = w_{k-1} - \alpha \widehat{\nabla} f(w_k)$ for some $\alpha > 0$. TODO. Learning rate schedulers, preconditioning, sampling with/without replacement, batch sizing, etc.

9.2 Momentum

Momentum

Momentum updates as follows, where g_k can be conceptualized as "negative velocity":

$$\begin{aligned} g_k &= \beta g_{k-1} - \nabla f(w_{k-1}) \\ w_k &= w_{k-1} - \alpha g_{k-1} \end{aligned}$$

It is equivalent to *heavy-ball momentum*, $w_k = w_{k-1} - \alpha \nabla f(w_{k-1}) + \beta(w_{k-1} - w_{k-2})$, and reduces to gradient descent when $\beta = 0$. Intuitively, in regions of low curvature, momentum mitigates stalled training, while in regions of high curvature, it smooths out fluctuations.

Remark. Algebraic equivalence to the heavy-ball form follows from taking

$$w_k = w_{k-1} - \alpha g_k \implies g_k = \frac{w_{k-1} - w_k}{\alpha}, \; g_{k-1} = \frac{w_{k-2} - w_{k-1}}{\alpha}$$

from which it follows

$$\begin{aligned} g_k = \beta g_{k-1} - \nabla f(w_{k-1}) &\implies \frac{w_{k-1} - w_k}{\alpha} = \beta \frac{w_{k-2} - w_{k-1}}{\alpha} - \nabla f(w_{k-1}) \\ &\implies w_k = w_{k-1} - \alpha \nabla f(w_{k-1}) + \beta(w_{k-1} - w_{k-2}) \end{aligned}$$

which is precisely the heavy-ball update. ■

Remark. The key insights of momentum are that it increases the convergence window for the step size α while speeding up convergence of ill-conditioned problems. The simplest nontrivial model is the convex quadratic, $f(w) = \frac{1}{2} w^\top A w - b^\top w$ for $A \succcurlyeq 0$ real-symmetric. Let's compare gradient descent and momentum. For gradient descent,

$$w_k = w_{k-1} - \alpha \nabla f(w_{k-1}) = w_{k-1} - \alpha A(w_{k-1} - w^*)$$

for unique minimizer $w^* = A^{-1} b$. Let $A = Q \Lambda Q^\top$ be the spectral decomposition with eigenvalues $\lambda_1 \geqslant \cdots \geqslant \lambda_n > 0$. Under a change of basis $v_k = Q^\top (w_k - w^*)$,

$$v_k = v_{k-1} - \alpha \Lambda Q^\top (w_{k-1} - w^*) = v_{k-1} - \alpha \Lambda v_{k-1} \implies v_k = (I - \alpha \Lambda)^k v_0$$

The convergence rate is therefore $\rho = \max_i |1 - \alpha \lambda_i| = \max\{|1 - \alpha \lambda_1|, |1 - \alpha \lambda_n|\}$. Notably, convergence occurs whenever $\rho < 1$, i.e. when $\alpha \lambda_1 < 2$. By a perturbation argument (if the rates are

not equal, you can decrease the larger one and increase the smaller one, thereby decreasing the maximum), the optimal step size α^* comes from equating the two rates,

$$|1-\alpha^*\lambda_1| = |1-\alpha^*\lambda_n| \implies 1-\alpha^*\lambda_1 = -1+\alpha^*\lambda_n \implies \alpha^* = \frac{2}{\lambda_1+\lambda_n},\ \rho^* = \frac{\kappa-1}{\kappa+1}$$

where $\kappa = \lambda_1/\lambda_n \geqslant 1$ is the condition number of A. In comparison, consider the momentum updates under a similar change of basis $v_k = Q^\top g_k$ and $x_k = Q^\top(w_k - w^*)$,

$$\begin{aligned} v_k &= \beta v_{k-1} + \Lambda x_{k-1} \\ x_k &= x_{k-1} - \alpha v_k = (I-\alpha\Lambda)x_{k-1} - \alpha\beta v_{k-1} \end{aligned}$$

This relation can be expressed as a linear recurrence,

$$\begin{bmatrix} v_k \\ x_k \end{bmatrix} = \begin{bmatrix} \beta I & \Lambda \\ -\alpha\beta I & I-\alpha\Lambda \end{bmatrix} \begin{bmatrix} v_{k-1} \\ x_{k-1} \end{bmatrix} \implies \begin{bmatrix} v_k(i) \\ x_k(i) \end{bmatrix} = \underbrace{\begin{bmatrix} \beta & \lambda_i \\ -\alpha\beta & 1-\alpha\lambda_i \end{bmatrix}}_{R_i} \begin{bmatrix} v_{k-1}(i) \\ x_{k-1}(i) \end{bmatrix} = R_i^k \begin{bmatrix} v_0(i) \\ x_0(i) \end{bmatrix}$$

In other words, the convergence in the ith eigenspace is governed by the spectral radius r_i of R_i, whose characteristic polynomial and spectrum are, respectively,

$$\begin{aligned} \chi_i(z) &= z^2 - (1-\alpha\lambda_i+\beta)z + \beta \\ \sigma_i &= \frac{1}{2}\left(1-\alpha\lambda_i+\beta \pm \sqrt{(1-\alpha\lambda_i+\beta)^2 - 4\beta}\right) \end{aligned}$$

Consider the discriminant $D_i = (1-\alpha\lambda_i+\beta)^2 - 4\beta$. If $D_i \leqslant 0$, then $\sqrt{D_i}$ is purely imaginary, so the spectrum is a singleton with magnitude $r_i = \sqrt{\beta} < 1$, and convergence always occurs. On the other hand if $D_i > 0$, the convergence criterion is $r_i < 1$; equivalently $\chi_i(z)$ must have roots inside the open unit disk. Checking boundary conditions $\chi_i(\pm 1) = 0$, we find $\alpha\lambda_i = 0$ and $\alpha\lambda_i = 2+2\beta$; the former trivially requires $\alpha > 0$, but the latter shows convergence for $\alpha\lambda_i < 2+2\beta$. Comparison with the $\alpha\lambda_i < 2$ bound for gradient descent shows that *momentum nearly doubles the effective window of convergence* whenever β is large!

Now we seek the optimal global convergence rate ρ^*, for which we must choose α^* and β^*. By a complex-valued analog to perturbation argument from gradient descent, the maximum convergence rate is minimized when

$$D_i \leqslant 0 \iff -2\sqrt{\beta} \leqslant 1-\alpha\lambda_i+\beta \leqslant 2\sqrt{\beta} \iff \lambda_i \in \left[\frac{(1-\sqrt{\beta})^2}{\alpha}, \frac{(1+\sqrt{\beta})^2}{\alpha}\right]$$

The optimum occurs when this window is just large enough, i.e.

$$\lambda_1 = \frac{(1+\sqrt{\beta^*})^2}{\alpha^*},\ \lambda_n = \frac{(1-\sqrt{\beta^*})^2}{\alpha^*} \implies \beta^* = \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2,\ \alpha^* = \left(\frac{2}{\sqrt{\lambda_1}+\sqrt{\lambda_n}}\right)^2$$

and the global convergence rate follows from $\rho^* = \sqrt{\beta^*}$. The insight is: *for ill-conditioned problems ($\kappa \gg 1$), momentum effectively square-roots the condition number* compared to gradient descent. ■

Part III

Convex Optimization

Strong Convexity

Let $f : K \to \mathbb{R}$ differentiable for $K \subseteq \mathbb{R}^n$. Then f is α-strongly convex if it has at least some curvature everywhere:

$$f(y) \geqslant f(x) + \nabla f(x)^\top (y - x) + \frac{1}{2}\alpha \|y - x\|^2$$

Informally, the function cannot be "too flat" anywhere, which is useful since small suboptimalities in output-space then automatically translate into to small suboptimalities in input-space: $f(x) - f(x^*) \geqslant \alpha \|x^* - x\|^2/2$. If $f \in C^2$, an equivalent characterization is that the spectrum of the Hessian is bounded from below by α, i.e. $\nabla^2 f(x) \geqslant \alpha I$.

Smoothness

Let $f : K \to \mathbb{R}$ differentiable for $K \subseteq \mathbb{R}^n$. Then f is β-smooth if its curvature is not too sharp anywhere:

$$f(y) \leqslant f(x) + \nabla f(x)^\top (y - x) + \frac{1}{2}\beta \|y - x\|^2$$

Equivalently, its gradient is β-Lipschitz, $\|\nabla f(y) - \nabla f(x)\| \leqslant \beta \|y - x\|$. Similarly, if $f \in C^2$, another characterization is $\nabla^2 f(x) \leqslant \beta I$.

Remark. If $f : K \to \mathbb{R}$ is α-strongly convex *and* β-smooth, then $\alpha I \preccurlyeq \nabla^2 f(x) \preccurlyeq \beta I$ so we have a local characterization of the function's spectrum. In this case we denote $\kappa = \beta/\alpha \geqslant 1$ the **condition number** of f, analogous to the matrix condition number $\kappa = \lambda_{\max}(A)/\lambda_{\min}(A)$ for $A > 0$. In fact, when $f(x) = \frac{1}{2}x^\top Ax$ for $A > 0$, these definitions agree. ■

Euclidean Projection

If $K \subseteq \mathbb{R}^n$ is convex and closed, then it has a Euclidean projection

$$\Pi_K(x) = \operatorname*{argmin}_{y \in K} \|y - x\|$$

The generalized Pythagorean theorem states that for such a set K,

$$\|y - x\| \geqslant |\|y - \Pi_K(x)\| \quad (y \in K, x \in \mathbb{R}^n)$$

10 Linear Programming

11 Convex Programming

12 Semidefinite Optimization

13 Online Convex Optimization

The OCO Protocol

The following defines the OCO protocol, which can be framed as a repeated game between a *decision maker* and an *adversary*. Notably, the losses may be adversarially selected for each round after knowing the decision maker's action.

Online Convex Optimization Protocol

```
Fix K convex and compact.
for t = 1, ..., T do
    Decision maker selects action x_t ∈ K according to algorithm A.
    Adversary reveals loss function f_t : K → ℝ.
    Decision maker incurs loss f_t(x_t).
end for
```

Because the losses are adversarial, there is no meaningful way to absolutely bound the cumulative loss of the decision maker. Instead, the relevant metric of performance is the performance against the best fixed action in hindsight, i.e. the *regret* function

$$\text{Regret}_T(\mathcal{A}) = \sum_{t=1}^{T} f_t(x_t) - \min_{x \in \mathcal{K}} \sum_{t=1}^{T} f_t(x) = \sum_{t=1}^{T} [f_t(x_t) - f_t(x^*)]$$

for $x^* \in \operatorname{argmin}_{x \in \mathcal{K}} \sum_t f_t(x)$. The goal is *sublinear regret* so that the average regret vanishes asymptotically, i.e. $\frac{1}{T}\text{Regret}_T(\mathcal{A}) \to 0$ as $T \to \infty$.

Remark. For this framework to make sense, several assumptions must be made:

- The adversary's losses must be uniformly bounded in time; otherwise, (s)he can assign an arbitrarily large loss to the decision-maker's first action (and provide zero loss to all other actions), making it impossible to bound regret from above.
- Compactness, with some regularity assumptions on f_t, prevents the adversary from setting aside some strategies with zero loss while increasing the decision-maker's losses indefinitely.

Surprisingly, much can be derived with not much more than these restrictions. ■

13.1 Prediction from Expert Advice

The Experts Setting

In this special case of OCO, $\mathcal{K} = \Delta_n$ is the probability simplex over n experts, each of whom presents a recommendation at time t represented by a basis vector $e_i \in \Delta_n$. Therefore the experts' loss vector is $l_t = (f_t(e_i))_{i=1}^n \in \mathbb{R}^n$.

Hedge Algorithm

The Hedge algorithm exponentially downweights the experts each round depending on their latest losses.

Hedge Algorithm

Initialize $W_1 = \mathbf{1}_n$.
for $t = 1, \ldots, T$ **do**
    Initialize distribution $x_t = \left[\frac{W_t(i)}{\sum_j W_t(j)}\right]_{i=1}^n$.
    Decision maker selects expert $i_t \sim x_t$.
    Adversary reveals loss function f_t, and DM incurs loss $l_t(i_t)$.
    Update weights $W_{t+1}(i) = W_t(i)\exp(-\eta l_t(i))$.
end for

14 Dynamical Systems

Part IV

Reinforcement Learning

15 Optimal Control

15.1 Bellman Equations

Stationary Policies

A *Markov* policy is one which is only dependent on the current state. A *stationary* policy is a Markov policy whose decisions are time-independent.

Value Functions

Fix an MDP $M = (\mathcal{S}, \mathcal{A}, \mathbb{P}, \mu, r, \gamma)$. The *state-value function* is $V^\pi : \mathcal{S} \to \mathbb{R}$, while the *action-value function* is $Q^\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. The functions are defined by (τ is a trajectory)

$$V^\pi(s) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t) \,\middle|\, s_0 = s\right]$$
$$Q^\pi(s,a) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t) \,\middle|\, s_0 = s, a_0 = a\right]$$

For simplicity of analysis, we often assume $T = \infty$. Note $\|V^\pi\|_\infty, \|Q^\pi\|_\infty \leqslant \frac{1}{1-\gamma}$, the latter of which is the *effective horizon*. They are connected by the *Bellman Consistency Equations*,

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q^\pi(s,a)]$$
$$Q^\pi(s,a) = r(s,a) + \gamma\mathbb{E}_{s'\sim\mathbb{P}(\cdot|s,a)}[V^\pi(s')]$$

16 Value Learning

16.1 TD-Learning

State-Value Iteration

Consider an agent which acts according to policy π. The goal of value learning is to predict the expected (discounted) future rewards for any current state of the agent.

Temporal Difference Learning

The Bellman operator specifies $(\mathcal{B}V)(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[r(s,a) + \gamma\mathbb{E}_{s'\sim\mathbb{P}(\cdot|s,a)}[V(s')]]$. TD($\lambda$) attempts to iteratively apply the Bellman operator using, effectively, the next $1/\lambda$ states. It tries to solve the *credit assignment* problem by sizing the TD-update using an *eligibility trace function*, which tracks the most recently used states at each time step.

The eligibility trace can either be *accumulated*, which is closer to the idea that frequent visitation should increase responsibility (but can increase instability), or *reset*, which is more conservative and may under-credit in some settings (but avoids trace explosion).

TD(λ) Algorithm

```
Initialize step size α, discount γ, trace parameter λ.
Initialize V(s) = e(s) = 0 for all states s ∈ S.
for episode = 1, 2, ... do
    Reset e(s) = 0 for all s ∈ S.
    for t = 0, 1, ..., T do
        Sample a_t ~ π(· | s_t); observe r(s_t, a_t) and s_{t+1}.
        Compute TD error δ ← r(s_t, a_t) + γV(s_{t+1}) − V(s_t).
        Decay the traces e(s) ← γλe(s) for all s ∈ S.
        Accumulate e(s_t) ← e(s_t) + 1, or replace e(s_t) ← 1.
        Update V(s) ← V(s) + αδe(s) for all s ∈ S.
    end for
end for
```

Remark. There is a smooth interpolation between $\lambda = 0$, which corresponds to high bias and low variance, and $\lambda = 1$ (Monte Carlo) for low bias but high variance. For TD(0), accumulating and resetting the eligibility trace agree. ■

16.2 SARSA and Q-Learning

SARSA and Expected SARSA

SARSA (State-Action-Reward-State-Action) is the on-policy analogue of TD(0) for the action-value function. At each step, the agent observes $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ and updates

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left(r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\right)$$

Since $a_{t+1} \sim \pi(\cdot \mid s_{t+1})$, SARSA evaluates the *behavior policy* including exploration (e.g.,

ε-greedy), making it on-policy. This causes SARSA to learn more conservative policies in stochastic environments, since it accounts for the cost of exploratory actions.

Expected SARSA replaces the sampled next action with an expectation over the policy,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_t + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s_{t+1})}[Q(s_{t+1}, a')] - Q(s_t, a_t) \right)$$

This eliminates the variance from sampling a_{t+1} while remaining on-policy. When π is greedy, Expected SARSA reduces to Q-learning; when π is ε-greedy, it provides a strictly lower-variance alternative to SARSA. In tabular settings, Expected SARSA converges under the same conditions as Q-learning but with empirically faster convergence.

Deep Q Networks (Mnih et. al., 2015)

DQN extends Q-learning to high-dimensional state spaces by approximating Q^* with a neural network Q_ϕ. Two key innovations stabilize training:

- **Experience replay.** Transitions (s_t, a_t, r_t, s_{t+1}) are stored in a replay buffer $\mathcal{D}$ and sampled uniformly at random for training. This breaks temporal correlations between consecutive samples and improves data efficiency.
- **Target network.** A separate target network Q_{ϕ^-} with parameters ϕ^- is used to compute TD targets, with ϕ^- updated slowly (either by periodic copying or Polyak averaging $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-$).

The loss is the mean squared Bellman error over minibatches from the replay buffer,

$$\mathcal{L}(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\phi^-}(s', a') - Q_\phi(s, a) \right)^2 \right]$$

The agent acts ε-greedily with respect to Q_ϕ, typically with ε annealed from 1.0 to 0.1.

Remark. **Notes.** Without target networks, the TD target $r + \gamma \max_{a'} Q_\phi(s', a')$ moves with every gradient step, creating a "chasing a moving target" phenomenon that causes oscillation or divergence. The target network provides a quasi-stationary objective over many updates.

Double DQN. Vanilla DQN suffers from *maximization bias*: the $\max_{a'} Q_{\phi^-}(s', a')$ operator systematically overestimates values since $\mathbb{E}[\max_i X_i] \geqslant \max_i \mathbb{E}[X_i]$. Double DQN (van Hasselt et. al., 2016) decouples action selection from evaluation,

$$y = r + \gamma Q_{\phi^-}(s', \operatorname*{argmax}_{a'} Q_\phi(s', a'))$$

The online network Q_ϕ selects the best action, but the target network Q_{ϕ^-} evaluates it. This significantly reduces overestimation and improves stability.

Issues. DQN is limited to discrete action spaces (the max over actions is intractable for continuous $\mathcal{A}$), does not naturally encourage exploration beyond ε-greedy, and the replay buffer introduces off-policy bias that can compound with function approximation errors (the "deadly triad" of function approximation, bootstrapping, and off-policy learning). ■

Rainbow (Hessel et. al., 2018)

Rainbow combines six orthogonal improvements to DQN into a single integrated agent:

- **Double Q-learning**: decouples action selection from evaluation to reduce overestimation bias.
- **Prioritized experience replay**: samples transitions with probability proportional to their TD error magnitude $|\delta_t|$, focusing learning on surprising or poorly-predicted transitions. Importance sampling weights correct for the induced bias.
- **Dueling networks**: decomposes the Q-function as $Q_\phi(s,a) = V_\psi(s) + A_\xi(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A_\xi(s,a')$, allowing the network to separately learn state value and action advantage. The centering ensures identifiability.
- **Multi-step returns**: replaces the one-step TD target with an n-step return $G_t^{(n)} = \sum_{k=0}^{n-1}\gamma^k r_{t+k} + \gamma^n \max_{a'} Q_{\phi^-}(s_{t+n}, a')$, propagating reward signal faster at the cost of some off-policy bias.
- **Distributional RL (C51)**: models the full return distribution $Z(s,a)$ rather than its expectation $Q(s,a) = \mathbb{E}[Z(s,a)]$, representing the distribution as a categorical over a fixed set of atoms and minimizing the projected KL divergence to the distributional Bellman target.
- **Noisy networks**: replaces ε-greedy exploration with learned stochastic linear layers $y = (\mu_w + \sigma_w \odot \varepsilon_w)x + \mu_b + \sigma_b \odot \varepsilon_b$, where ε is sampled noise. The noise parameters σ are learned, enabling state-dependent exploration that adapts over training.

The ablation study in the original paper demonstrates that each component contributes positively, with distributional RL and prioritized replay providing the largest individual gains. Rainbow achieved state-of-the-art performance on the Atari-57 benchmark at the time of publication.

17 Policy Optimization

Policy Optimization

Model the policy using a neural network $\pi_\theta : \mathcal{S} \to \mathcal{A}$. Then the problem becomes

$$\underset{\theta}{\text{maximize}} \quad J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{s\sim\mu}[V^{\pi_\theta}(s)]$$

where $\tau = (s_0, a_0, r_0, \ldots, s_T, a_T, r_T)$ is a trajectory, $R_t(\tau) = \sum_{t'\geqslant t} \gamma^{t'-t} r(s_{t'}, a_{t'})$ is a one-step estimate of $Q^{\pi_\theta}(s_t, a_t)$, and μ is the initial distribution. By convention, we set $R(\tau) = R_0(\tau)$. The optimizer is trained using gradient ascent,

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\pi_\theta)$$

for some appropriate learning rate $\alpha > 0$.

Remark. Advantages compared to DQN include a more efficient approach to learning continuous action spaces and encouraging more exploration (not just epsilon-greedy). Many policy gradient-methods are on-policy, allowing the model to directly optimize for its evaluated behavior.

Note that the last equality in the objective follows by conditioning on the initial state,

$$\mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \mathbb{E}_{s\sim\mu}[\mathbb{E}_{\tau\sim\pi_\theta|s_0=s}[R(\tau)]] = \mathbb{E}_{s\sim\mu}[V^{\pi_\theta}(s)]$$

We typically use the latter characterization when analyzing infinite time horizons since V^{π_θ} can be manipulated using the Bellman equations. ■

Policy Gradients

Evaluating $\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$ is generally a difficult problem because the transition dynamics $\mathbb{P}(s' \mid s, a)$ are unknown. However, it turns out that

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t R_t(\tau) \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t Q^{\pi_\theta}(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

where the last equality follows from the identity $Q^{\pi_\theta}(s_t, a_t) = \mathbb{E}_{\tau\sim\pi_\theta}[R_t(\tau) \mid s_t, a_t]$. Notably, $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ can be found using modern auto-differentiation tools. So, we can estimate the policy gradient by averaging this object over several trajectories.

Remark. A straightforward proof is as follows via the **log-derivative trick**. For any function $f(x)$ and likelihood $p(x;\theta)$,

$$\begin{aligned}
\nabla_\theta \mathbb{E}_{x\sim p(x;\theta)}[f(x)] &= \nabla_\theta \int f(x) p(x;\theta)\, dx = \int f(x) \nabla_\theta p(x;\theta)\, dx \\
&= \int f(x) p(x;\theta) \frac{\nabla_\theta p(x;\theta)}{p(x;\theta)}\, dx = \int f(x) \nabla_\theta \log p(x;\theta) p(x;\theta)\, dx \\
&= \mathbb{E}_{x\sim p(x;\theta)}[f(x) \nabla_\theta \log p(x;\theta)]
\end{aligned}$$

and thus $\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)\nabla_\theta \log p(\tau;\theta)]$. So we just need to differentiate the log-likelihood. Observe that if μ is the initial state distribution,

$$p(\tau;\theta) = \mu(s_0)\prod_{t=0}^{T}\mathbb{P}(s_{t+1} \mid s_t, a_t)\pi_\theta(a_t \mid s_t)$$
$$\implies \log p(\tau;\theta) = \log \mu(s_0) + \sum_{t=0}^{T}(\log \mathbb{P}(s_{t+1} \mid s_t, a_t) + \log \pi_\theta(a_t \mid s_t))$$

so $\nabla_\theta \log p(\tau;\theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$, where $\nabla_\theta \log \pi_\theta(a \mid s)$ is the **score function**. Thus,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\sum_{t=0}^{T}\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] = \sum_{t=0}^{T}\sum_{t'=0}^{T}\gamma^{t'}\mathbb{E}_{\tau\sim\pi_\theta}[r_{t'}\nabla_\theta \log \pi_\theta(a_t \mid s_t)]$$

Note that for $t' < t$, conditioning on the history until immediately before a_t is selected allows us to remove $r_{t'}$ from the conditional expectation:

$$\mathbb{E}_{\tau\sim\pi_\theta}[r_{t'}\nabla_\theta \log \pi_\theta(a_t \mid s_t)] = \mathbb{E}_{\tau\sim\pi_\theta}[r_{t'}\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)]]$$

But it turns out that the **expectation of the score function is zero**:

$$\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)] = \int_{\mathcal{A}} \pi_\theta(a_t \mid s_t)\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, da_t = \nabla_\theta \underbrace{\int_{\mathcal{A}} \pi_\theta(a_t \mid s_t) da_t}_{=1} = 0$$

So only terms for which $t' \geqslant t$ survive in the policy gradient; hence

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\sum_{t'=t}^{T}\left(\gamma^{t'}r_{t'}\right)\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t R_t(\tau)\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

which is precisely as desired. ■

Discounted State Occupancy Measure (DSOM)

It is a probability mass $d^{\pi_\theta} : \mathcal{S} \to [0,1]$ representing the total discounted proportion of time spent in state s following policy π_θ,

$$d^{\pi_\theta}(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^t \mathbb{P}^{\pi_\theta}[s_t = s]$$

Importantly, for any function $f(s_t, a_t)$, we have the **occupancy measure transform**,

$$\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t f(s_t, a_t)\right] = \underbrace{(1-\gamma)^{-1}}_{\substack{\text{effective}\\ \text{horizon}}}\underbrace{\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}[f(s,a)]}_{\text{expected } f \text{ under DSOM}}$$

An important special case arises from our existing identities for $J(\theta)$ and $\nabla_\theta J(\theta)$:

$$J(\theta) = (1-\gamma)\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}[r(s,a)]$$
$$\nabla_\theta J(\theta) = (1-\gamma)\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}[A^{\pi_\theta}(s,a)\nabla_\theta \log \pi_\theta(a \mid s)]$$

The DSOM can be extended to continuous state spaces as well. Notably, the expressions for $J(\theta)$ and $\nabla_\theta J(\theta)$ remain identical.

Remark. It's straightforward to prove the transformation in the discrete case:

$$\begin{aligned}
\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t f(s_t,a_t)\right] &= \sum_{t=0}^{\infty}\gamma^t\sum_{s\in\mathcal{S}}\mathbb{P}^{\pi_\theta}(s_t=s)\sum_{a\in\mathcal{A}}\pi(a\mid s)f(s,a)\\
&= \sum_{s\in\mathcal{S}}\left(\sum_{t=0}^{\infty}\gamma^t\mathbb{P}^{\pi_\theta}(s_t=s)\right)\sum_{a\in\mathcal{A}}\pi(a\mid s)f(s,a)\\
&= (1-\gamma)\sum_{s\in\mathcal{S}}d^{\pi_\theta}(s)\sum_{a\in\mathcal{A}}\pi(a\mid s)f(s,a)\\
&= (1-\gamma)\mathbb{E}_{s\sim d^{\pi_\theta},a\sim\pi(\cdot|s)}[f(s,a)]
\end{aligned}$$

In the gradient expression, we can replace $Q^{\pi_\theta}(s,a) = A^{\pi_\theta}(s,a) + V^{\pi_\theta}(s)$ with $A^{\pi_\theta}(s,a)$ since

$$\mathbb{E}_{s\sim d^{\pi_\theta},a\sim\pi_\theta(\cdot|s)}[V^{\pi_\theta}(s)\nabla_\theta\log\pi_\theta(a\mid s)] = \mathbb{E}_{s\sim d^{\pi_\theta}}[V^{\pi_\theta}(s)\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta\log\pi_\theta(a\mid s)]] = 0$$

and in fact, **adding any state function $b(s)$ does not change the bias**. ■

Importance Sampling

For likelihoods p_1, p_2 and function f satisfying $p_1 f \ll p_2$ (absolute continuity),

$$\mathbb{E}_{x\sim p_1}[f(x)] = \int_{\mathcal{X}} f(x)p_1(x)\,dx = \int_{\mathcal{X}}\frac{p_1(x)}{p_2(x)}f(x)p_2(x)\,dx = \mathbb{E}_{x\sim p_2}\left[\frac{p_1(x)}{p_2(x)}f(x)\right]$$

The likelihood ratio $p_1(x)/p_2(x)$ is called the *importance weight*. Informally, $x\sim p_2$ "cancels" with $p_2(x)$ and is replaced by $x\sim p_1$.

17.1 REINFORCE

REINFORCE (Williams, 1992)

The key idea is to estimate the policy gradient using one-sample Monte-Carlo:

REINFORCE Algorithm

```
Initialize learning rate α
Initialize weights θ of a policy network π_θ
for episode = 1, 2, ... do
    Sample a trajectory τ = (s_0, a_0, r_0, ..., s_T, a_T, r_T) ~ π_θ
    Set ∇_θ J(θ) ← 0
    for t = 0, 1, ..., T do
        R_t(τ) ← Σ_{t'=t}^{T} γ^{t'-t} r_{t'}
        ∇_θ J(θ) ← ∇_θ J(θ) + R_t(τ) ∇_θ log π_θ(a_t | s_t)
    end for
    θ ← θ + α ∇_θ J(θ)
end for
```

Remark. **Notes.** Each trajectory's experiences are discarded after each use; REINFORCE is an on-policy algorithm. Some ways to reduce the variance of the algorithm include subtracting a state-dependent baseline,

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T} (R_t(\tau) - b(s_t)) \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

One option for b is the value function V^π, which motivates the Actor-Critic algorithm. Another is the average (state-independent) reward-to-date in the trajectory, $b = \frac{1}{T}\sum_{t=0}^{T} R_t(\tau)$, which encourages half of the better actions and discouraging half of the worse actions. This can be helpful if, for instance, all of the actions give negative rewards, but some moreso than others.

Issues. A bad policy (e.g. if exploration is low) leads to bad data, which leads to worse gradient estimates, creating a cycle that leads to performance collapse. REINFORCE is very unstable! ■

17.2 Natural Policy Gradients

Fisher Information Matrix

It is the covariance matrix of the score function, or equivalently the negative expected Hessian of the log-likelihood,

$$\begin{aligned} F(\pi_\theta) &= \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)} \left[\nabla_\theta \log \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)^\top\right] \\ &= \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)} \left[-\nabla_\theta^2 \log \pi_\theta(a \mid s)\right] \end{aligned}$$

where the second equality is known as the **Fisher information identity**. $F(\pi_\theta)$ is also the Hessian of the expected conditional KL divergence $F(\pi_\theta) = \nabla_{\theta'}^2 \widetilde{\mathbb{D}}_{\mathrm{KL}}[\pi_\theta \parallel \pi_{\theta'}]|_{\theta'=\theta}$, where

$$\begin{aligned} \widetilde{\mathbb{D}}_{\mathrm{KL}}[\pi_\theta \parallel \pi_{\theta'}] &= \mathbb{E}_{s\sim d^{\pi_\theta}}[\mathbb{D}_{\mathrm{KL}}[\pi_\theta(\cdot \mid s) \parallel \pi_{\theta'}(\cdot \mid s)]] \\ &= \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}[\log \pi_\theta(a \mid s) - \log \pi_{\theta'}(a \mid s)] \end{aligned}$$

Since the score has zero expectation, it locally defines the geometry of the *statistical manifold*:

$$\widetilde{\mathbb{D}}_{\mathrm{KL}}[\pi_\theta \parallel \pi_{\theta+\Delta\theta}] \approx \underbrace{\widetilde{\mathbb{D}}_{\mathrm{KL}}[\pi_\theta \parallel \pi_\theta]}_{=0} + \frac{1}{2}\Delta\theta^\top F(\pi_\theta)\Delta\theta = \frac{1}{2}\Delta\theta^\top F(\pi_\theta)\Delta\theta$$

(A statistical manifold is a family of distributions $\mathcal{M} = \{p(x;\theta) : \theta \in \Theta\}$ for *parameter space* $\Theta \subseteq \mathbb{R}^n$, with local coordinates θ^i and *Fisher-Rao metric* $g_{ij}(\theta) = F(\pi_\theta)_{ij}$).

Remark. In this context, we care about the Fisher information identity primarily since it allows us to connect $F(\pi_\theta)$ to the KL divergence. Indeed,

$$\begin{aligned} \nabla_{\theta'}^2 \widetilde{\mathbb{D}}_{\mathrm{KL}}[\pi_\theta \parallel \pi'_\theta]|_{\theta'=\theta} &= \nabla_{\theta'}^2 \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}[\log \pi_\theta(a \mid s) - \log \pi_{\theta'}(a \mid s)] \\ &= \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}[-\nabla_{\theta'}^2 \log \pi_{\theta'}(a \mid s)] = F(\pi_\theta) \end{aligned}$$

So, why is the identity true? Intuitively, for some parametrized distribution $p(x;\theta)$, the Hessian $-\nabla_\theta^2 \log p(x;\theta)$ measures how sensitive the log-likelihood is to θ: if it is large (e.g. in operator norm), then the model will sharply degrade under perturbation of θ. On the other hand, $\nabla_\theta \log p(x;\theta) \nabla_\theta p(x;\theta)^\top$ measures the variance of the gradient of log-likelihood.

Therefore, the identity says that on average, if the log-likelihood is very sensitive to θ, then its gradient must fluctuate a lot. Formally,

$$\int_{\mathcal{X}} p(x;\theta)\,dx = 1 \implies \int_{\mathcal{X}} \nabla_\theta p(x;\theta)\,dx = \int_{\mathcal{X}} (\nabla_\theta \log p(x;\theta))p(x;\theta)\,dx = 0$$

Differentiating with respect to θ,

$$\int_{\mathcal{X}} [(\nabla_\theta^2 \log p(x;\theta))p(x;\theta) + \nabla_\theta p(x;\theta)\nabla_\theta \log p(x;\theta)^\top]\,dx = 0$$

Applying the log-derivative trick again yields the identity. ■

Natural Policy Gradient (Kakade, 2001)

The natural policy gradient defines the optimal search direction subject to constraints in the KL divergence; equivalently, it is the direction of steepest ascent on the statistical manifold defined by the Fisher-Rao metric. It is the (approximate) solution to the following convex optimization, which itself is a first-order expansion of the objective change $\Delta J(\theta)$:

$$\begin{aligned} \underset{\Delta\theta}{\text{maximize}} \quad & \nabla_\theta J(\theta)^\top \Delta\theta \\ \text{subject to} \quad & \widetilde{\mathbb{D}}_{\mathrm{KL}}[\pi_\theta \parallel \pi_{\theta+\Delta\theta}] \leqslant \delta \end{aligned}$$

The optimal step can be found by a Taylor approximation of the KL divergence (using the Fisher information matrix) and enforcing the KKT conditions; its direction is

$$\widetilde{\nabla}_\theta J(\theta) = F(\pi_\theta)^{-1}\nabla_\theta J(\theta)$$

with optimal step size

$$\alpha = \sqrt{\frac{2\delta}{\nabla_\theta J(\theta)^\top F(\pi_\theta)^{-1}\nabla_\theta J(\theta)}}$$

The natural policy gradient update is $\theta \leftarrow \theta + \alpha\widetilde{\nabla}_\theta J(\theta)$.

Remark. The natural policy gradient is motivated by these issues with vanilla policy gradients:

- A small step in parameter space can lead to massive distributional shifts (large KL divergence) when the policy is in a sensitive region, leading to *catastrophic performance collapse*.
- Conversely, in flat regions of parameter space, it may take several updates to improve a policy, leading to slow training.

This phenomenon arises because the vanilla policy gradient is *parametrization dependent*. For instance, a linear reparametrization of θ would change the step size and direction: with new coordinates $\phi = A\theta$, the gradient becomes $\nabla_\phi J(\pi_\phi) = A^{-\top}\nabla_\theta J(\theta)$. So, the effective updates are

$$\begin{aligned} \phi_{t+1} &= \phi_t + \alpha A^{-\top}\nabla_\theta J(\theta) \\ \theta_{t+1}^{\text{eff}} &= A^{-1}\phi_{t+1} = \theta_t + \alpha(A^\top A)^{-1}\nabla_\theta J(\theta) \end{aligned}$$

so the step is dependent on parametrization unless A is orthogonal. Informally, the issue is that $\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta)$ is dimensionally inconsistent since $\nabla_\theta J(\theta) = (\partial J/\partial\theta_i)_i$ has dimensions θ_i^{-1}.

Natural policy gradients are limited in practice since inversion of the Fisher information matrix at each iteration is extremely costly at $\mathcal{O}(|\theta|^3)$, especially for neural networks with millions or billions of parameters; additionally, it may be that some steps blow up the KL divergence if the Taylor approximation is not good (e.g., if the Hessian is not positive-definite). However, the ideas form the basis of TRPO, which itself is the foundation of modern policy optimization. ■

17.3 Conservative Policy Iteration

The Performance Difference Identity

The *performance difference* between two policies π_θ and $\pi_{\theta'}$ is

$$J(\theta') - J(\theta) = \mathbb{E}_{\tau\sim\pi_{\theta'}}\left[\sum_{t=0}^{T}\gamma^t A^{\pi_\theta}(s_t, a_t)\right] = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_{\theta'}}, a\sim\pi_{\theta'}(\cdot|s)}[A^{\pi_\theta}(s,a)]$$

So, the difference is the average π_θ-advantage sampled from $\pi_{\theta'}$. Note that it vanishes when the policies are equivalent, as expected.

Remark. The identity is a result of the Bellman consistency equations,

$$\begin{aligned}\mathbb{E}_{\tau\sim\pi_{\theta'}}\left[\sum_{t=0}^{T}\gamma^t A^{\pi_\theta}(s_t, a_t)\right] &= \mathbb{E}_{\tau\sim\pi_{\theta'}}\left[\sum_{t=0}^{T}\gamma^t (r(s_t,a_t) + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t))\right] \\ &= \mathbb{E}_{\tau\sim\pi_{\theta'}}[R(\tau) - V^{\pi_\theta}(s_0)] = J(\pi'_\theta) - J(\theta)\end{aligned}$$

The penultimate step follows since the last two series telescope. Note that the final step is only true under the MDP assumption that the initial state distribution μ is set by the environment. ■

Conservative Policy Iteration (Kakade & Langford, 2002)

CPI provides monotonic improvement guarantees for *mixture* policies as a way to address catastrophic performance collapse. It does this by introducing a *surrogate objective*,

$$L_{\pi_\theta}(\pi_{\theta'}) = J(\theta) + \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)}A^{\pi_\theta}(s,a)\right]$$

Often we drop the $J(\theta)$ and $(1-\gamma)^{-1}$ since they don't impact the optimization. Notably $\nabla_\theta J(\theta) = \nabla_{\theta'} L_{\pi_\theta}(\pi_{\theta'})|_{\theta'=\theta}$, so $L_{\pi_\theta}(\pi_{\theta'})$ and $J(\theta')$ agree to first-order. If the update returns a *mixture policy* $\pi_{\theta'} = (1-\alpha^*)\pi_\theta + \alpha^* \operatorname{argmax}_\pi L_{\pi_\theta}(\pi)$ for some optimal mixing parameter $\alpha^* \in [0,1]$ defined by

$$\alpha^* = \operatorname*{argmax}_\alpha\left(\alpha\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_{\theta'}(\cdot|s)}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)}A^{\pi_\theta}(s,a)\right] - \frac{2\gamma\varepsilon\alpha^2}{(1-\gamma)^2}\right)$$

where $\varepsilon = \max_s |\mathbb{E}_{a\sim\pi_{\theta'}(\cdot|s)}[A^{\pi_\theta}(s,a)]|$, then the *CPI lower bound* is satisfied,

$$J(\theta') \geqslant L_{\pi_\theta}(\pi_{\theta'}) - \frac{2\gamma\varepsilon\alpha^2}{(1-\gamma)^2} = J(\theta) + \alpha\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)}A^{\pi_\theta}(s,a)\right] - \frac{2\gamma\varepsilon\alpha^2}{(1-\gamma)^2}$$

Thus for small enough α, monotonic improvement is guaranteed, since the performance difference is bounded below by $c_1\alpha - c_2\alpha^2$.

Remark. **Notes.** Since maximizing the objective is equivalent to maximizing the performance difference, let's compare this quantity with the surrogate: by importance sampling,

$$J(\theta') - J(\theta) = \mathbb{E}_{s\sim d^{\pi_{\theta'}}, a\sim\pi'_\theta(\cdot|s)}\left[A^{\pi_\theta}(s,a)\right] = \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}\left[\frac{d^{\pi_{\theta'}}(s)}{d^{\pi_\theta}(s)}\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)}A^{\pi_\theta}(s,a)\right]$$

So, the surrogate is making the approximation $\frac{d^{\pi_{\theta'}}(s)}{d^{\pi_\theta}(s)} \approx 1$.

Issues. The two largest problems are that monotonic improvement is only guaranteed for mixture policies (which deep neural networks violate) and that the constant c_2 is extraordinarily large, thus requiring a massive surrogate objective to guarantee positive improvement. ■

17.4 Trust-Region Policy Optimization

Trust Region Policy Optimization (Schulman et. al., 2015)

TRPO provides a *theory of monotonic improvement for general policy update rules*. It frames the problem as a contrained optimization, building on the ideas of CPI and NPG:

$$\begin{aligned} \underset{\theta'}{\text{maximize}} \quad & \widetilde{L}_{\pi_\theta}(\pi_{\theta'}) = \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}\left[\frac{\pi_{\theta'}(a \mid s)}{\pi_\theta(a \mid s)}A^{\pi_\theta}(s,a)\right] \\ \text{subject to} \quad & \widetilde{\mathbb{D}}_{\mathrm{KL}}[\pi_\theta \parallel \pi_{\theta'}] \leqslant \delta \end{aligned}$$

As with natural policy gradients, a first-order approximation reduces the problem to

$$\begin{aligned} \underset{\Delta\theta}{\text{maximize}} \quad & (\underbrace{\nabla_{\theta'} L_{\pi_\theta}(\pi_{\theta'})|_{\theta'=\theta}}_{g_\theta})^\top \Delta\theta \\ \text{subject to} \quad & \frac{1}{2}\Delta\theta^\top F(\pi_\theta)\Delta\theta \leqslant \delta \end{aligned}$$

The ideal direction is $s_\theta = F(\pi_\theta)^{-1} g_\theta$ which is computed by the conjugate-gradient method (requiring only $\mathcal{O}(|\theta|)$ per matvec thanks to **Pearlmutter's trick**!). The step size is computed by backtracking line search, starting from the natural policy gradient step size (the optimal size assuming the conjugate-gradient solution and first-order approximation are exact):

$$\theta' = \theta + \alpha^j \sqrt{\frac{2\delta}{{s_\theta}^\top F(\pi_\theta) s_\theta}} s_\theta = \theta + \alpha^j \sqrt{\frac{2\delta}{{g_\theta}^\top s_\theta}} s_\theta$$

where $j \in \{0, \ldots, J\}$ is the smallest positive integer such that both the *performance difference is positive* and the *KL divergence constraint is satisfied*. If no such $j \leqslant J$ exists, then the policy remains constant while more data is collected at the next iteration. (For the analysis, take $J = \infty$). TRPO guarantees that

$$J(\theta') \geqslant L_{\pi_\theta}(\pi_{\theta'}) - \frac{4\varepsilon\gamma}{(1-\gamma)^2}\max_{s\in\mathcal{S}} \mathbb{D}_{\mathrm{KL}}[\pi_\theta(\cdot \mid s) \parallel \pi_{\theta'}(\cdot \mid s)]$$

Notably, $\pi_{\theta'}$ does *not* have to be a mixture policy.

Remark. **Notes.** Pearlmutter's trick computes Hessian-vector products using only two forward/back-

ward passes and thus in $\mathcal{O}(n)$ time! Suppose $f : \mathbb{R}^n \to \mathbb{R}$ and $v \in \mathbb{R}^n$; we seek $\nabla^2 f(x)v$. Note

$$\nabla f(x)^\top v = \sum_{j=1}^{n} \frac{\partial f}{\partial x_j} v_j \implies \nabla(\nabla f(x)^\top v) = \left(\sum_{j=1}^{n} \frac{\partial^2 f}{\partial x_i \partial x_j} v_j \right)_{1 \leqslant i \leqslant n} = \nabla^2 f(x) v$$

So, we require only two gradient operations to compute the product, making each iteration of conjugate gradient $\mathcal{O}(n)$.

Issues. There are three salient issues.

- The quadratic approximation of the KL divergence is still fragile; if the geometry is highly nonlinear, line search may repeatedly reject updates, leading to stalled training.
- Backtracking line search and conjugate gradient (even with Pearlmutter's trick) are computationally expensive.
- Actor-critic methods, which share parameters between π_θ and V^{π_θ}, cannot be used since the actor (policy network) is updated by a constrained optimization, while the critic (value network) is unconstrained. So, even mixing a value update with a policy update can violently upset the local geometry.

As it turns out, proximal policy optimization will abandon the second-order Fisher information matrix computation and the hard KL constraint, addressing these issues. ■

17.5 Proximal Policy Optimization

Generalized Advantage Estimation (Schulman et. al., 2016)

GAE provides a family of advantage estimators that interpolate between bootstrapped and Monte Carlo estimates. Recall the TD error $\delta_t = r(s_t, a_t) + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$. Note δ_t is an unbiased one-step estimate of the advantage, $\mathbb{E}_{\tau\sim\pi_\theta}[\delta_t \mid s_t, a_t] = A^{\pi_\theta}(s_t, a_t)$. The *generalized advantage estimate* is an exponentially-weighted average of multi-step advantage estimates:

$$\widehat{\mathbb{A}}_t = \sum_{t'=t}^{T} (\gamma\lambda)^{t'-t}\delta_{t'} \quad (\lambda \in [0,1])$$

where λ is the *bias-variance tradeoff parameter*, and conventionally $0^0 = 1$. Notably, if $\lambda = 0$, then $\widehat{\mathbb{A}} = \delta_t$, and if $\lambda = 1$, then $\widehat{\mathbb{A}}_t = R_t(\tau) - V(s_t)$, the Monte Carlo advantage.

Proximal Policy Optimization (Schulman et. al., 2017)

PPO replaces TRPO's constrained optimization with a *clipped surrogate objective* that implicitly enforces a trust region, achieving comparable or superior empirical performance at dramatically lower implementation and computational cost. Define the probability ratio

$$\rho_t(\theta') = \frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}$$

The actor loss is the **PPO-Clip** objective; for some clipping hyperparameter $\varepsilon > 0$,

$$\mathcal{L}^{\text{CLIP}}(\theta') = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \min\left(\rho_t(\theta')\widehat{\mathbb{A}}_t,\ \text{clip}(\rho_t(\theta'), 1-\varepsilon, 1+\varepsilon)\widehat{\mathbb{A}}_t\right)\right]$$

The critic loss is the usual sum-of-squares error of the value function,

$$\mathcal{L}^{\text{VF}}(\phi) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}(V_\phi(s_t) - R_t(\tau))^2\right]$$

Finally, we define the **entropy bonus for exploration**,

$$\mathcal{H}(\theta') = \mathbb{E}_{s\sim d^{\pi_\theta}}[H(\pi_{\theta'}(\cdot \mid s))]$$

The full PPO loss, combining the actor, critic, and an entropy bonus for exploration, is

$$\boxed{\mathcal{L}^{\text{PPO}}(\theta', \phi) = -\mathcal{L}^{\text{CLIP}}(\theta') + c_1\mathcal{L}^{\text{VF}}(\phi) - c_2\mathcal{H}(\theta')}$$

for $\alpha, \beta > 0$ weighting coefficients.

Remark. **Mechanics of Clipping.** By case analysis on the sign of $\widehat{\mathbb{A}}_t$:

- If $\widehat{\mathbb{A}}_t > 0$ (action was better than expected): increasing ρ_t improves the surrogate up to a

factor of $1+\varepsilon$. The minimum selects the more conservative term, so

$$\min(\rho_t \widehat{\mathbb{A}}_t, (1+\varepsilon)\widehat{\mathbb{A}}_t) = \min(\rho_t, 1+\varepsilon)\widehat{\mathbb{A}}_t$$

preventing the policy from moving too aggressively toward good actions.

- If $\widehat{\mathbb{A}}_t < 0$ (action was worse than expected): decreasing ρ_t improves the surrogate up to a factor of $1-\varepsilon$. Here,

$$\min(\rho_t \widehat{\mathbb{A}}_t, (1-\varepsilon)\widehat{\mathbb{A}}_t) = \max(\rho_t, 1-\varepsilon)\widehat{\mathbb{A}}_t$$

preventing the policy from moving too far from bad actions.

In both cases, the gradient is zero outside the trust region $\rho_t \in [1-\varepsilon, 1+\varepsilon]$, providing a "soft wall" analogous to TRPO's hard KL constraint. Crucially, the clipped objective is a *pessimistic* lower bound on the unclipped surrogate, so PPO never exploits overconfident advantage estimates.

Notes. We discuss several important details.

- *Minibatch updates.* Unlike TRPO (which performs a single constrained step per batch of data), PPO performs multiple epochs of minibatch SGD on the same batch of trajectories. This is the primary source of its sample efficiency gains. Clipping ensures that each epoch does not stray too far from the data-collecting policy, even without an explicit constraint.
- *Entropy bonus.* The term $c_2\mathcal{H}(\pi_{\theta'})$ encourages exploration by penalizing deterministic policies. Without it, the policy can prematurely converge to a suboptimal deterministic action. This is especially important in environments with sparse or deceptive rewards.
- *Value function clipping.* In some implementations (e.g., the original PPO paper), the value function loss is also clipped:

$$\mathcal{L}^{\text{VF,clip}}(\phi) = \mathbb{E}\left[\max\left((V_\phi(s_t) - R_t)^2,\ (V_{\phi_{\text{old}}}(s_t) + \text{clip}(\Delta V, -\varepsilon, \varepsilon) - R_t)^2\right)\right]$$

where $\Delta V = V_\phi(s_t) - V_{\phi_{\text{old}}}(s_t)$. However, subsequent empirical studies have found this can hurt performance, and it is often omitted.

- *Shared parameters.* Since both the actor and critic losses are unconstrained and differentiable, they can be combined into a single loss function and optimized jointly on a shared network backbone. This resolves the fundamental incompatibility between TRPO and actor-critic architectures noted earlier.

Comparison with TRPO. The core simplification is summarized as follows:

	TRPO	**PPO-Clip**
Constraint	Hard KL divergence	Implicit via clipping
Optimization	Conjugate gradient + line search	Minibatch SGD
Second-order	Fisher information matrix	Not required
Updates per batch	1	Multiple epochs
Actor-Critic	Incompatible (shared params)	Compatible

PPO achieves competitive or better empirical results by trading theoretical guarantees for practical robustness. Unlike TRPO, PPO does not come with a formal monotonic improvement bound; the clipping is a heuristic that works well in practice. ■

PPO-Clip Algorithm

The full algorithm operates as follows:

PPO-Clip Algorithm

1: Initialize policy parameters θ, value function parameters ϕ
2: Initialize hyperparameters: learning rate α, clip range ε, GAE parameter λ, number of epochs K, minibatch size M
3: **for** $iteration = 1, 2, \ldots$ **do**
4: Collect a batch of trajectories $\mathcal{D} = \{\tau_i\}$ using π_θ
5: Compute GAE advantages $\widehat{\mathbb{A}}_t$ for all $(s_t, a_t) \in \mathcal{D}$ using V_ϕ
6: Normalize advantages: $\widehat{\mathbb{A}}_t \leftarrow (\widehat{\mathbb{A}}_t - \mu_{\widehat{\mathbb{A}}})/\sigma_{\widehat{\mathbb{A}}}$ over the batch
7: **for** $epoch = 1, \ldots, K$ **do**
8: **for** each minibatch $\mathcal{B} \subset \mathcal{D}$ of size M **do**
9: Compute ratios $\rho_t(\theta') = \pi_{\theta'}(a_t \mid s_t)/\pi_\theta(a_t \mid s_t)$ for all $t \in \mathcal{B}$
10: Compute clipped surrogate: $\mathcal{L}^{\text{CLIP}} = \frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}} \min(\rho_t \widehat{\mathbb{A}}_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\widehat{\mathbb{A}}_t)$
11: Compute value loss: $\mathcal{L}^{\text{VF}} = \frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}}(V_\phi(s_t) - R_t)^2$
12: Compute entropy bonus: $\mathcal{H} = \frac{1}{|\mathcal{B}|}\sum_{t\in\mathcal{B}} H(\pi_{\theta'}(\cdot \mid s_t))$
13: $(\theta', \phi) \leftarrow (\theta', \phi) - \alpha\nabla_{(\theta',\phi)}(-\mathcal{L}^{\text{CLIP}} + c_1\mathcal{L}^{\text{VF}} - c_2\mathcal{H})$
14: **end for**
15: **end for**
16: $\theta \leftarrow \theta'$
17: **end for**

Remark. **Notes.** Advantage normalization (Line 6) is a critical implementation detail: it centers the advantages to have zero mean and unit variance across the batch, ensuring that roughly half the actions are reinforced and half are suppressed, regardless of the absolute scale of returns. This is conceptually related to the average reward baseline discussed in REINFORCE.

Standard hyperparameters (from the original paper and subsequent best practices) are $\varepsilon = 0.2$, $\lambda = 0.95$, $\gamma = 0.99$, $K = 3$–10 epochs, and $c_1 = 0.5$, $c_2 = 0.01$. The number of epochs K must be tuned carefully: too few epochs waste data, while too many can degrade performance as the ratio ρ_t drifts and the advantage estimates become stale.

PPO-Penalty. An alternative formulation replaces clipping with a soft KL penalty,

$$\mathcal{L}^{\text{KL}}(\theta') = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \rho_t(\theta')\widehat{\mathbb{A}}_t\right] - \beta\widetilde{\mathbb{D}}_{\text{KL}}[\pi_\theta \parallel \pi_{\theta'}]$$

where $\beta > 0$ is an adaptive penalty coefficient. When the KL divergence exceeds 1.5δ, β is doubled; when it falls below $\delta/1.5$, β is halved. While this variant is closer to TRPO in spirit (directly penalizing distributional shift), PPO-Clip generally outperforms it empirically, suggesting that the pessimistic clipping mechanism is a more robust proxy for trust region enforcement than a tuned penalty. ■

17.6 Group-Relative Policy Optimization

TODO.

18 Actor-Critic Methods

Actor-Critic

Roughly speaking, actor-critic methods bring together the main families thus far: Q-learning and policy optimization. An actor-critic architecture maintains two components:

- **Actor** π_θ: the policy network, updated by policy gradient methods.
- **Critic** V_ϕ: a value function approximator with parameters ϕ, trained by regression on observed returns.

The critic provides advantage estimates (e.g., via GAE) that are used in place of Monte Carlo returns in the policy gradient,

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}\gamma^t \widehat{\mathbb{A}}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

The critic is updated by minimizing the squared error loss

$$\mathcal{L}(V_\phi) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T}(V_\phi(s_t) - R_t(\tau))^2\right]$$

Remark. **Notes.** The baseline subtraction used in REINFORCE can be viewed as a special case: using $b(s) = V^{\pi_\theta}(s)$ gives the advantage $A^{\pi_\theta}(s,a) = Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s)$, which is exactly what the critic estimates. The key distinction is that in actor-critic, the critic is actively learned and used for bootstrapping (via TD residuals), not just as a passive baseline.

Issues. When parameters are shared between the actor and critic (e.g. in a common feature extractor), constrained policy updates like TRPO become problematic: the trust region governs the actor, but the critic's unconstrained regression updates can perturb the shared representations, violating the local geometry assumptions. PPO's clipping mechanism, as we will see, is far more amenable to shared architectures. ■

18.1 Determinstic Policy Gradients

Determinstic Policy Gradient Theorem

In continuous control, it's often desirable for the actor to output a fixed action $\mu_\theta(s)$, not a distribution $\pi_\theta(\cdot \mid s)$. Why?

- *Variance reduction:* the stochastic gradient $\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s) Q(s,a)]$ depends on sampling actions a, which can be very noisy.
- *High-dimensional complexity:* if actions are high-dimensional, the stochastic policy has to model a distribution whose complexity can scale with dimension, while the deterministic one needs to only output a single action.

Deep Deterministic Policy Gradients (Lillicrap et al., 2015)

DDPG generalizes Q-learning to continuous state and action spaces.

DDPG Algorithm

Initialize networks μ, μ^-, Q, Q^-, replay buffer $\mathcal{D} \leftarrow \varnothing$.
for *episode* $= 1, 2, \ldots$ **do**
  Initialize stochastic noise process Z_t.
  Choose action $a_t \leftarrow \mu(s_t) + Z_t$, observe r_t, s_{t+1}.
  Update replay buffer $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t, a_t, r_t, s_{t+1})$.
  Sample minibatch uniformly $\{(s_i, a_i, r_i, s_i')\}_{i=1}^N \subseteq \mathcal{D}$.
  Define the TD targets $y_i \leftarrow r_i + \gamma Q^-(s_i', \mu^-(s_i'))$.
  Minimize the critic loss $\mathcal{L}(\theta^Q) = \mathrm{MSE}(\{Q(s_i, a_i), y_i\}_{i=1}^N)$.
  Maximize the actor objective $J(\theta^\mu) = \frac{1}{N}\sum_{i=1}^N Q(s_i, \mu(s_i))$.
  Perform Polyak update $\theta^{\mu^-} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu^-}$.
  Perform Polyak update $\theta^{Q^-} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q^-}$.
end for

The replay buffer is generally fixed size (and kicks out the oldest transition once full), and the noise process is generally Gaussian or Ornstein-Uhlenbeck. For stability, the target networks are slowly updated with $\tau \in [10^{-3}, 10^{-2}]$.

Remark. DDPG has three main issues:

- *Overestimation Bias:* suppose $Q^-(s,a) = Q^-_{\text{true}}(s,a) + \delta Q(s,a)$ for some error function δQ. Then the actor will systematically learn to exploit actions with high noise.
- *Instability:* DDPG combines bootstrapping, neural function approximation, and off-policy replay. All are hard on their own, and together they result in extreme brittleness.
- *Shifting Targets:* if the critic network updates too quickly, learning can become unstable, since the actor is always trying to match a moving target. This may lead to performance collapse, as in REINFORCE, due to a loop of bad exploration and bad actions.

As we'll see, TD3 addresses each of these issues. ■

Twin Delayed DDPG (Fujimoto et al., 2018)

TD3 presents three main changes to DDPG:

- *Twin critics* Q_1 and Q_2; the actor is optimized conservatively against the min of their estimates. Due to different initializations, they in general will not agree.
- *Delayed actor updates* every d steps (usually $d = 2$) allow the critic to "settle" before the actor optimizes against it.
- *Target policy smoothing* introduces clipped noise to the target policy, forcing the actor

to only learn actions *robust* to the noise. More precisely, the update becomes

$$\begin{aligned} a_i' &= \mu^-(s_i') + \text{clip}(\varepsilon_i, -c, c) \quad (\varepsilon_i \sim \mathcal{N}(0, \sigma)) \\ y_i &= r_i + \gamma Q^-(s_i', a_i') \end{aligned}$$

18.2 Soft Actor-Critic

Maximum Entropy RL

MaxEnt reinforcement learning is a framework which rewards entropy at each time step:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi_\theta(\cdot \mid s_t))) \right]$$

Note that this framework is distinct from entropy as a loss regularizer (e.g. PPO), which is more to keep the policy diffuse, avoid premature collapse, and encourage exploration. In particular, MaxEnt encourages the agent to *plan ahead* to reach states with high entropy.

Soft-Actor Critic (Haarnoja et al., 2018)

It is an off-policy actor-critic algorithm within the MaxEnt framework, TODO.

19 Goal-Conditioning

19.1 Universal Value Function Approximators

Universal Value Function Approximators (Schaul et al. 2015)

Given a goal space $\mathcal{G}$, define a a pseudo-reward function $r(s, a, g)$. Define general value functions that represent the expected pseudo-return:

$$V^{\pi_\theta}(s, g) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, g) \,\middle|\, s_0 = s\right] : \mathcal{S} \times \mathcal{G} \to \mathbb{R}$$
$$Q^{\pi_\theta}(s, a, g) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, g) \,\middle|\, s_0 = s, a_0 = a\right] : \mathcal{S} \times \mathcal{A} \times \mathcal{G} \to \mathbb{R}$$

The Bellman equations are also satisfied:

$$V^{\pi_\theta}(s, g) = \mathbb{E}_{a \sim \pi_\theta(\cdot|s)}[Q^{\pi_\theta}(s, a, g)]$$
$$Q^{\pi_\theta}(s, a, g) = r(s, a, g) + \gamma \mathbb{E}_{s' \sim \mathbb{P}(\cdot|s,a)}[V^{\pi_\theta}(s', g)]$$

Remark. The motivating question: in what theoretical framework can an agent learn how to value states for many different goals at once? For example, maybe a Minecraft agent has several goals: 1) make bacon, 2) kill a zombie, 3) trade with a villager. It is inefficient to treat these as three distinct reward functions, since the there is shared environmental knowledge which helps with each goal. The key contribution of UVFA is to develop a framework in which these goals are therefore treated as inputs to the same learned value function. ■

19.2 Hindsight Experience Replay

Hindsight Experience Replay (Andrychowicz et al., 2017)

The idea of HER is that for sparse reward functions in the goal-conditioned setting, we can add *pseudo-goals* to our replay buffer so that the agent learns about environment dynamics even without a reward. Formally, we assume

- To each goal g corresponds a *success function* $f_g : \mathcal{S} \to \{0, 1\}$; that is, $f_g(s) = 1$ if and only if state s successfully achieves goal g.
- Given any s, there exists $m(s) \in \mathcal{G}$ such that $f_{m(s)}(s) = 1$ (so $m : \mathcal{S} \to \mathcal{G}$). (We want to be able to introduce pseudo-goals no matter which state we're in.)

HER is a drop-in modification to off-policy algorithms which already have a replay buffer. In the following, $\cdot \| \cdot$ denotes concatenation.

Hindsight Experience Replay

```
Initialize replay buffer D ← ∅.
for episode = 1, 2, ... do
    Sample g from goal distribution, τ from the off-policy algorithm.
    for t = 0, 1, ... do
```

5: Store $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t \| g, a_t, r_t, s_{t+1} \| g)$.
6: Store $\mathcal{D} \leftarrow \mathcal{D} \cup (s_t \| g', a_t, r_t, s_{t+1} \| g')$ for each pseudo-goal g'.
7: **end for**
8: Perform optimizer updates according to the off-policy algorithm.
9: **end for**

Remark. How do we select good pseudo-goals g'? The simplest method is to choose only $m(s_T)$, the goal associated with our terminal state. However, experiments show it can be better to choose $k \approx 4 - 8$ pseudo-goals uniformly from the *future* of that episode. In practical implementations, whenever an experience is selected from the replay buffer, it has probability $p_{\text{HER}} = 1 - \frac{1}{k+1} = \frac{k}{k+1}$ of being swapped out for a (future) transition. In this sense, the hyperparameter k controls the ratio of pseudo to real goals in the buffer. ■

Remark. Example: consider a locomotion task over $\mathcal{S} = \mathcal{G} = \mathbb{R}$ with a deterministic goal distribution; that is, we always "sample" $g = g_0 \in \mathbb{R}$ in Line 3 of the Hindsight Experience Replay algorithm. The success functions then are $f_g(s) = \mathbb{I}_{\{g\}}(s)$. ■

19.3 Contrastive Reinforcement Learning

Contrastive Learning as Goal-Conditioned RL (Eysenbach et al., 2022)

20 Unsupervised Skill Discovery

20.1 Noise-Contrastive Estimation

The methods in the previous sections (DIAYN, MISL, CIC) all rest on a mutual information objective whose key computational challenge is estimating the partition function of a density model. Noise-Contrastive Estimation (NCE) provides a principled way to sidestep this by converting density estimation into binary classification.

Noise-Contrastive Estimation (Gutmann & Hyvärinen, 2010)

Let p_θ be an unnormalized model with intractable partition function $Z(\theta)$. Given a *noise distribution* q, NCE draws k noise samples per real datum and trains a binary classifier to distinguish real data from noise. The optimal log-odds ratio of the classifier is

$$\log \frac{P(\text{real} \mid x)}{P(\text{noise} \mid x)} = \log p_\theta(x) - \log q(x) - \log k$$

so a well-trained classifier implicitly recovers $\log p_\theta$ without ever evaluating $Z(\theta)$. The training objective is binary cross-entropy over real/noise pairs:

$$\mathcal{L}_{\text{NCE}}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}}[\log \sigma(f_\theta(x))] + k \cdot \mathbb{E}_{x \sim q}[\log(1 - \sigma(f_\theta(x)))]$$

where $f_\theta(x) = \log p_\theta(x) - \log q(x) - \log k$ and σ is the sigmoid. The partition function is either absorbed as a learned scalar bias or held fixed.

Remark. **Why classification instead of likelihood?** The partition function $Z(\theta) = \int p_\theta(x)\, dx$ requires a sum or integral over all of input space—intractable for large vocabularies or continuous domains. NCE replaces this global normalization with a local, sample-based comparison: at each step, the model only needs to score $k + 1$ candidates, not the entire space.

Connection to negative sampling. Word2Vec's skip-gram negative sampling is a special case of NCE with a unigram noise distribution. Each positive (context, word) pair is contrasted against k randomly drawn words; the shared weights learn to embed words such that co-occurring pairs score high.

Effect of k. Larger k provides a better approximation to true MLE but at greater computational cost. With $k = 1$, NCE reduces to a balanced binary classifier, and with $k \to \infty$, the bias toward p_{data} dominates and the estimator approaches maximum likelihood. ■

20.2 Contrastive Predictive Coding

NCE converts density estimation into binary classification. Contrastive Predictive Coding (CPC) extends this idea in a new direction: rather than fitting a density model, it trains a representation by asking whether a context vector can *predict* future observations—without modeling their full distribution.

CPC (van den Oord et al., 2018)

Given a sequential input $x_1, x_2, \ldots$, CPC learns:

- A *local encoder* g_{enc} mapping each x_t to a latent $z_t = g_{\text{enc}}(x_t)$.
- An *autoregressive model* g_{ar} (e.g., a GRU) summarizing $z_1, \ldots, z_t$ into a context c_t.

For each prediction horizon k, a linear head W_k predicts the future representation via $\hat{z}_{t+k} = W_k c_t$. The model is trained with the **InfoNCE** loss: given one positive z_{t+k} and N negatives $\{z_j\}_{j=1}^N$ drawn from other sequences,

$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E}\left[\log \frac{\exp(z_{t+k}^\top W_k c_t)}{\exp(z_{t+k}^\top W_k c_t) + \sum_{j=1}^N \exp(z_j^\top W_k c_t)}\right]$$

This is a softmax cross-entropy over an $(N+1)$-way identification problem: which of the $N+1$ candidates is the true future? After training, the encoder g_{enc} and context model g_{ar} are transferred to downstream tasks; the prediction heads W_k are discarded.

Remark. **InfoNCE as a mutual information lower bound.** The InfoNCE loss is a lower bound on the mutual information between the future z_{t+k} and the context c_t:

$$I(z_{t+k};\, c_t) \geqslant \log(N+1) - \mathcal{L}_{\text{InfoNCE}}$$

Minimizing the loss therefore *maximizes a lower bound on* $I(z_{t+k}; c_t)$, forcing the context to retain exactly the information about the past that is predictive of the future—and discard the rest (unpredictable noise, irrelevant details).

What is learned. The encoder is not trained to reconstruct x_t (a generative objective) but to produce representations such that future states can be *identified* from the current context. This naturally induces a bottleneck: representations that are too noisy cannot support identification, while representations that are too compressed cannot distinguish positives from hard negatives.

Connection to NCE. InfoNCE is a multi-class generalization of NCE: binary NCE (one negative) becomes an $(N+1)$-way classification problem with N negatives. As $N \to \infty$, the lower bound on $I(z_{t+k}; c_t)$ tightens and the objective approaches the true mutual information.

Connection to RL. The CPC framework is formally close to the intrinsic objectives of DIAYN and CIC: in all three cases, a representation is trained so that pairing a context (skill z or past context c_t) with a state enables identification in a contrastive pool. CPC makes the temporal structure explicit—the "skill" is the past, and the "state" is the future. ■

21 Contrastive Reinforcement Learning

The policy gradient and value-based methods developed so far all share a common assumption: the reward function $r(s, a)$ is given to the agent, and the task is to maximize the expected discounted sum of rewards. In many practical settings, however, reward functions are either absent, expensive to specify, or so sparse that the agent cannot learn from them effectively. This motivates a family of approaches that attempt to learn *representations* or *intrinsic objectives* that are useful for downstream tasks—without relying on dense, hand-specified rewards.

Contrastive reinforcement learning sits at the intersection of two ideas that we now develop from scratch: *contrastive representation learning*, which extracts structure from observations by comparing them, and *mutual information as an intrinsic reward*, which gives the agent an objective even when the environment provides no signal. After building these foundations, we will see that they are not merely loosely related heuristics—they are, under certain conditions, formally equivalent to learning optimal value functions.

21.1 Contrastive Learning

Contrastive Learning

Consider observation space $\mathcal{X}$. Given *anchor* $x \in \mathcal{X}$, *positive* $x^+ \in \mathcal{X}$ (semantically similar to x), and K *negatives* $\{x_k^-\}_{k=1}^K$ (semantically dissimilar), a contrastive loss trains an encoder $f_\theta : \mathcal{X} \to \mathbb{R}^d$ so that $f_\theta(x)$ is close to $f_\theta(x^+)$ and far from each $f_\theta(x_k^-)$.

The canonical instance is the **InfoNCE loss** (also called NT-Xent or momentum contrast), which can be derived as maximizing a lower bound on the mutual information $I(x; x^+)$:

$$\mathcal{L}_{\text{NCE}}(\theta) = -\mathbb{E}\left[\log \frac{\exp(f_\theta(x)^\top f_\theta(x^+)/\tau)}{\exp(f_\theta(x)^\top f_\theta(x^+)/\tau) + \sum_{k=1}^K \exp(f_\theta(x)^\top f_\theta(x_k^-)/\tau)}\right]$$

where $\tau > 0$ is a *temperature* hyperparameter. The loss is minimized when the encoder assigns high cosine similarity to positive pairs and low similarity to negative pairs.

Remark. **Why mutual information?** Intuitively, $I(x; x^+) = \mathcal{H}(x^+) - \mathcal{H}(x^+ \mid x)$ is large if knowing x tells us a lot about x^+. Maximizing $I(x; x^+)$ therefore encourages the encoder to capture the shared, semantically meaningful structure between x and its positive counterpart. The InfoNCE loss provides a lower bound on this mutual information: since the log-sum-exp denominator upper-bounds the true partition function of the unnormalized distribution $\exp(f_\theta(x)^\top f_\theta(\cdot)/\tau)$, minimizing $\mathcal{L}_{\text{NCE}}$ pushes up a tractable estimate of $I(x; x^+)$.

Positive pair construction. The utility of contrastive learning depends critically on what counts as a "positive". In computer vision, positives are typically *augmented views* of the same image (crops, color jitters, flips). In RL, the most natural choice for temporally structured data is *states visited near each other in time*, as we will see throughout this section.

Temperature. τ controls the sharpness of the distribution. Small τ concentrates probability mass on the hardest negative (most similar to the anchor), encouraging fine-grained discrimination but potentially destabilizing training. Large τ softens the comparison and is more forgiving. ■

21.2 CURL: Contrastive Representation Learning for RL

CURL (Laskin et al., 2020)

The core difficulty of applying deep RL to pixel-based observations (e.g., raw image inputs from Atari or DMControl) is that the observation space is high-dimensional, noisy, and contains many irrelevant features (background textures, lighting). CURL (*Contrastive Unsupervised Representations for Reinforcement Learning*) addresses this by augmenting any model-free RL algorithm (such as SAC or Rainbow) with a contrastive auxiliary loss that trains the encoder to produce compact, task-relevant representations.

Concretely, given observation o_t, CURL generates two augmented views via random crops:

$$q = f_\theta(\text{aug}(o_t)), \quad k^+ = f_{\bar{\theta}}(\text{aug}(o_t))$$

where f_θ is the *query encoder* and $f_{\bar{\theta}}$ is a *momentum encoder* with parameters $\bar{\theta} = m\bar{\theta} + (1 - m)\theta$; that is, updated by Polyak averaging with $m \approx 0.999$. Negatives $\{k_i^-\}$ are drawn from other observations in the minibatch. The contrastive loss uses a learned bilinear similarity $\text{sim}(q, k) = q^\top W k$ rather than raw cosine similarity, giving:

$$\mathcal{L}_{\text{CURL}} = -\log \frac{\exp(q^\top W k^+ / \tau)}{\exp(q^\top W k^+ / \tau) + \sum_i \exp(q^\top W k_i^- / \tau)}$$

The total training objective is $\mathcal{L}_{\text{RL}} + \lambda \mathcal{L}_{\text{CURL}}$, where $\mathcal{L}_{\text{RL}}$ is the standard RL loss (e.g., the SAC objective or DQN Bellman error) and the encoder f_θ is shared.

Remark. **Why a momentum encoder?** Without it, both q and k^+ are functions of the same parameters θ, and naive gradient descent collapses: the encoder can minimize the loss by mapping all observations to the same representation. The momentum encoder creates a slowly-moving, stable target. This is directly analogous to the *target network* in DQN, which we saw earlier was necessary for the same reason.

What CURL achieves. By encouraging the encoder to produce similar representations for different augmented views of the same observation, the auxiliary loss implicitly enforces *augmentation invariance*: the representation should capture content (the agent's location, task-relevant objects), not style (exact pixel values, crop offsets). This allows the RL agent to learn from far fewer environment interactions when operating on pixels.

Limitations and context. CURL treats contrastive learning as a *representation-learning auxiliary*: it helps the encoder but does not change the reward or the objective. The agent still needs environment reward to learn task-relevant behavior. The next set of methods go further, using mutual information to *replace* or *supplement* the reward signal. ■

21.3 Mutual Information as an Intrinsic Reward

We now shift from using contrastive methods to improve observation encoders to using them to generate intrinsic reward signals. The key idea is to define an objective purely in terms of the agent's trajectory—without any external reward—by asking: *how much information does the agent's behavior convey?*

Mutual Information in RL

Let z be a latent variable (a "skill" or "goal") drawn from a prior $p(z)$, and let s be a state visited by the agent when executing the behavior associated with z. The *mutual information between z and s* is

$$I(s;z) = H(z) - H(z \mid s) = H(s) - H(s \mid z)$$

We can interpret these two decompositions as follows:

- $H(z) - H(z \mid s)$: the skill z should be *identifiable* from the state s. Given a visited state, an observer should be able to infer which skill the agent was using (low $H(z \mid s)$), and the skills themselves should be diverse (high $H(z)$).
- $H(s) - H(s \mid z)$: each skill should produce a *distinct distribution over states* (high $H(s)$), and conditioning on the skill should reduce uncertainty about which states are visited (low $H(s \mid z)$).

Maximizing $I(s;z)$ gives the agent an unsupervised objective that drives it to explore diverse, distinguishable behaviors without any hand-designed reward.

Remark. **The role of the lower bound.** The mutual information $I(s;z)$ is generally intractable to compute exactly, since it requires integrating over the state distribution $p(s \mid z)$ induced by the agent's policy. In practice, we use a *variational lower bound*: introduce a learned discriminator $q_\phi(z \mid s)$ (the "posterior" over skills given the state), and write

$$I(s;z) = H(z) - H(z \mid s) \geqslant H(z) + \mathbb{E}_{s,z}[\log q_\phi(z \mid s)]$$

using the fact that $H(z \mid s) \leqslant -\mathbb{E}[\log q_\phi(z \mid s)]$ for any q_ϕ (this is an instance of the general inequality $\mathbb{D}_{\mathrm{KL}}[p \parallel q] \geqslant 0$ applied to the conditional). We can then maximize this lower bound by jointly training the policy and discriminator. ■

21.4 Diversity is All You Need (DIAYN)

DIAYN (Eysenbach et al., 2019)

DIAYN (*Diversity is All You Need*) instantiates the mutual information objective to learn a diverse set of skills in a completely unsupervised way—no task reward is used during training.

Setup. Fix a categorical prior $p(z) = \mathrm{Uniform}(\{1, \ldots, K\})$ over K discrete skills. A skill-conditioned policy $\pi_\theta(a \mid s, z)$ is trained alongside a discriminator $q_\phi(z \mid s)$. The intrinsic reward at each step is defined as

$$r^z(s,z) = \log q_\phi(z \mid s) - \log p(z)$$

This is the *log ratio* of the posterior to the prior. Intuitively, the agent is rewarded for visiting states that allow an observer to determine *which skill* is being executed.

The training procedure alternates between:

1. **Policy update:** Update θ using any off-the-shelf RL algorithm (e.g., SAC) with intrinsic reward $r^z(s,z)$.

2. **Discriminator update:** Update ϕ by maximizing the log-likelihood $\mathbb{E}_{s,z}[\log q_\phi(z \mid s)]$ over transitions collected by the policy.

At the end of training, the learned skills can be composed or fine-tuned on downstream tasks with sparse environment reward.

Remark. **Why does this objective produce useful skills?** To see this, observe that maximizing $\mathbb{E}[\log q_\phi(z \mid s) - \log p(z)]$ is equivalent to maximizing the variational lower bound on $I(s;z)$:

$$\mathbb{E}_{z\sim p(z), s\sim p(s|z)}[\log q_\phi(z \mid s) - \log p(z)] = \mathbb{E}_{z,s}[\log q_\phi(z \mid s)] + H(z)$$

which is exactly the lower bound on $I(s;z) = H(z) + \mathbb{E}[\log p(z \mid s)]$ from the previous section (replacing the true posterior $p(z \mid s)$ with the learned approximation $q_\phi(z \mid s)$). So, DIAYN trains the policy to maximize a tractable proxy for skill-state mutual information.

What skills does DIAYN learn? In practice, the skills tend to correspond to distinct locomotion behaviors (walking, hopping, spinning) or distinct regions of the state space. The diversity emerges because different skills "compete" for different states: if skill $z = 1$ visits state s, then the discriminator is trained to output $q_\phi(z = 1 \mid s) \approx 1$. Skill $z = 2$ is then penalized for visiting s (it would receive a low reward $\log q_\phi(z = 2 \mid s) \approx \log 0$), so it is pushed toward visiting different states.

Entropy regularization. DIAYN is typically paired with maximum-entropy RL (e.g., SAC), which adds an entropy bonus $\alpha H(\pi(\cdot \mid s, z))$ to encourage the policy to be stochastic. This prevents collapse to deterministic behaviors and improves exploration.

Limitations. DIAYN uses a discrete, fixed set of skills and a discriminator over single states, which means it may fail to discover skills that are only distinguishable by their *trajectory* rather than any single visited state. We will see two remedies for this: CIC addresses the representation quality of the discriminator, while MISL addresses the trajectory-level formulation. ■

21.5 Mutual Information Skill Learning (MISL)

DIAYN uses a tabular or feedforward discriminator that maps individual states to skill posteriors. This is limiting: two skills that visit the same distribution of individual states but follow different *paths* through state space cannot be distinguished. MISL addresses this by enriching the mutual information objective to capture trajectory-level structure.

MISL (Hartikainen et al., 2019)

MISL (*Mutual Information Skill Learning*) formulates skill discovery as maximizing the mutual information between the *trajectory* of the agent and the skill variable, rather than just the final state:

$$I(\tau; z) = H(z) - H(z \mid \tau)$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is the full trajectory. Since operating on full trajectories is computationally expensive, MISL typically conditions the mutual information objective on a trajectory *summary statistic* (e.g., the initial and final state) or uses a recurrent discriminator $q_\phi(z \mid \tau)$.

The intrinsic reward is again derived from the variational lower bound, now applied to the trajectory-level posterior:

$$r^z(\tau, z) = \log q_\phi(z \mid \tau) - \log p(z)$$

The policy is updated to maximize the discounted sum of these intrinsic rewards, and the discriminator is updated via supervised learning on collected trajectory–skill pairs.

Remark. **Trajectory-level discrimination enables finer skill distinctions.** Consider two skills that both navigate a maze but one takes a clockwise path and the other counterclockwise. Both visit similar sets of individual states, so a state-only discriminator cannot distinguish them. A trajectory-level discriminator, however, can observe the *order* of state visitations and correctly identify each skill.

Practical implementation. A common simplification is to condition on a pair (s_0, s_T) rather than the full trajectory, approximating $I(s_T \mid s_0; z)$. This captures *which final state was reached from which initial state*—a notion of "goal-directedness" that intermediate-state discrimination misses—while remaining computationally tractable.

Connection to goal-conditioned RL. The factorization $I(s_T \mid s_0; z)$ bears a striking resemblance to the goal-conditioned RL setup, where the agent is rewarded for reaching a target state g from an initial state s_0. We will formalize this connection later by showing that contrastive methods can directly implement goal-conditioned value functions. ■

21.6 Contrastive Intrinsic Control (CIC)

DIAYN and MISL both decompose the mutual information objective into a policy learning phase and a discriminator learning phase. A natural question is: *can we use contrastive learning—rather than a softmax classifier—to implement the discriminator?* CIC answers yes, and shows that doing so leads to better-structured, more continuous skill spaces.

CIC (Laskin et al., 2022)

CIC (*Contrastive Intrinsic Control*) replaces the discrete skill posterior $q_\phi(z \mid s)$ of DIAYN with a contrastive representation. The key observation is that the mutual information lower bound

$$I(s; z) \geqslant H(z) + \mathbb{E}_{s,z}[\log q_\phi(z \mid s)]$$

can be maximized using an InfoNCE-style objective instead of a cross-entropy classifier, enabling *continuous* skill spaces $z \in \mathbb{R}^d$.

Architecture. CIC maintains:

- A *state encoder* $g_\theta : \mathcal{S} \to \mathbb{R}^d$ that maps states to a latent space.
- A *skill encoder* $h_\psi : \mathcal{Z} \to \mathbb{R}^d$ that maps skills (continuous vectors) to the same latent space.

A skill-state pair (z, s) is treated as a *positive pair* if s was actually visited by the policy under skill z. Pairs (z, s') from different skill-trajectory combinations form negatives. The

contrastive loss is

$$\mathcal{L}_{\text{CIC}} = -\mathbb{E}\left[\log \frac{\exp(g_\theta(s)^\top h_\psi(z)/\tau)}{\sum_{(s',z')\in\mathcal{N}} \exp(g_\theta(s')^\top h_\psi(z')/\tau)}\right]$$

The intrinsic reward is the *particle-based entropy estimate* of the skill-conditioned state distribution, approximating $H(s \mid z)$ by measuring how spread out the embeddings $g_\theta(s)$ are for fixed z.

Remark. **Continuous skills and the skill manifold.** Unlike DIAYN's discrete categories, CIC's continuous skill space $\mathcal{Z} = \mathbb{R}^d$ allows for *smooth interpolation* between behaviors: skills close in embedding space should produce similar behaviors, enabling structured exploration and hierarchical control.

Particle-based entropy. A key technical contribution of CIC is an efficient estimator for the state coverage entropy $H(s \mid z)$. Rather than fitting an explicit density model, CIC uses a k-nearest-neighbor entropy estimator on the embeddings $\{g_\theta(s_i)\}$: the entropy is approximated by measuring the average log-distance to the nearest neighbors, which is high when embeddings are spread out (good coverage) and low when they are clustered (poor coverage).

Synthesis. CIC can be seen as a synthesis of CURL and DIAYN: it borrows the contrastive, augmentation-based training of CURL to learn a representation, and uses that representation to implement the discriminator of DIAYN, but with a continuous skill space. This makes it a natural bridge to the next result, which shows that contrastive representations can directly encode value functions. ■

21.7 Contrastive RL as Goal-Conditioned RL

The methods above treat contrastive learning as a way to build representations or discriminators that are *used by* an RL agent. Eysenbach et al. (2022) establish a deeper connection: under the right choice of contrastive objective and positive/negative pair construction, contrastive learning *directly solves* a goal-conditioned RL problem. The representation that emerges is not merely a useful feature—it *is* the optimal value function.

Goal-Conditioned Reinforcement Learning

In goal-conditioned RL (GCRL), the agent is rewarded for reaching a goal state $g \in \mathcal{S}$. The reward is typically $r(s,g) = -\mathbf{1}[s \neq g]$ (negative distance) or $r(s,g) = \mathbf{1}[s = g]$ (indicator). The goal-conditioned value function is

$$V^\pi(s,g) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r(s_t,g) \,\middle|\, s_0 = s\right]$$

and the goal-conditioned Q-function $Q^\pi(s,a,g)$ is defined analogously. The optimal policy is $\pi^*(a \mid s,g) = \operatorname{argmax}_a Q^*(s,a,g)$.

Contrastive RL (Eysenbach et al., 2022)

Contrastive RL (*Contrastive Learning as Goal-Conditioned RL*) proves that the Q-function of a goal-conditioned agent with geometric temporal discounting has a contrastive structure. Specifically, consider the successor representation (SR) framework: define the *discounted state occupancy* between s and g as

$$C(s,a,g) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^t p(s_t = g \mid s_0 = s, a_0 = a)$$

which is the total discounted probability of reaching g from (s,a). When the reward is the indicator $r(s_t,g) = \mathbf{1}[s_t = g]$, we have $Q^\pi(s,a,g) \propto C^\pi(s,a,g)$.

The key insight is that $C^\pi(s,a,g)$ has a **contrastive factorization**. Let $\phi_\theta : \mathcal{S}\times\mathcal{A} \to \mathbb{R}^d$ be a representation of (s,a) pairs and $\psi_\theta : \mathcal{S} \to \mathbb{R}^d$ a representation of goal states. Define

$$\hat{C}_\theta(s,a,g) = \sigma\Big(\phi_\theta(s,a)^\top \psi_\theta(g)\Big)$$

for sigmoid σ. Then the following contrastive objective is equivalent to temporal difference learning of C^π:

$$\mathcal{L}_{\text{CRL}} = -\mathbb{E}\left[\log \hat{C}_\theta(s,a,g^+) + \log(1-\hat{C}_\theta(s,a,g^-))\right]$$

where **positive pairs** (s,a,g^+) are constructed by sampling g^+ as a *future state actually visited by the agent from* (s,a): specifically, $g^+ = s_{t+k}$ for $k \sim \text{Geom}(1-\gamma)$. **Negative pairs** (s,a,g^-) sample g^- from the marginal state distribution d^π independent of (s,a).

Remark. **Why future states as positives?** The geometric sampling $k \sim \text{Geom}(1-\gamma)$ is precisely the distribution over future states induced by the γ-discounted Bellman operator. Therefore, a state g sampled by this procedure is *more likely to appear as a positive if it is frequently reachable from* (s,a), exactly matching the definition of $C^\pi(s,a,g)$. In other words, contrastive training with temporally structured positives *implements Bellman backup* in representation space.

Binary cross-entropy as TD learning. The loss $\mathcal{L}_{\text{CRL}}$ is a binary cross-entropy (logistic regression) loss with positive examples being future states and negative examples being random states. The sigmoid output $\hat{C}_\theta(s,a,g) \in (0,1)$ can be interpreted as the probability that g is a future state of (s,a)—and this is exactly the normalized Q-value $Q^\pi(s,a,g)/Q^{\max}$. So, the representation $\phi_\theta(s,a)^\top\psi_\theta(g)$ implicitly encodes the optimal value function: *learning a representation is learning a value function*.

Policy extraction. Given $\hat{C}_\theta$, the greedy policy is

$$\pi(a \mid s,g) = \operatorname*{argmax}_a \hat{C}_\theta(s,a,g) = \operatorname*{argmax}_a \phi_\theta(s,a)^\top\psi_\theta(g)$$

This is computationally cheap: policy improvement is just a dot-product maximization in the representation space, without any Bellman iteration at test time.

Connections to prior work. This result unifies and explains the heuristics used in DIAYN and CIC. In both methods, the discriminator $q_\phi(z \mid s)$ was trained with a contrastive-like objective and used to generate intrinsic rewards. Contrastive RL reveals the precise sense in which this is principled: the discriminator is approximating the discounted future state occupancy $C^\pi(s,z)$, which *is* the Q-function of the corresponding goal-conditioned problem. ■

Copy

21.8 Goal-Conditioned Reinforcement Learning

The preceding subsection introduced goal-conditioned RL (GCRL) as a formal framework and showed that contrastive learning can directly solve it. Here we develop GCRL as an independent subject: what makes it fundamentally different from standard RL, what challenges arise, and what algorithmic ideas have been proposed to address them. The treatment draws heavily on the survey of Liu et al. (2022), which provides a systematic taxonomy of the field.

Goal-Augmented Markov Decision Process

Standard RL operates on an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, r, \gamma, \rho_0 \rangle$, where the reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is fixed and the agent learns a single policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$. GCRL generalizes this by augmenting the MDP with a *goal space* $\mathcal{G}$, a *goal distribution* p_g, and a *goal-mapping function* $\phi : \mathcal{S} \to \mathcal{G}$ that projects states onto the goal space. The resulting **goal-augmented MDP** (GA-MDP) is the tuple $\langle \mathcal{S}, \mathcal{A}, T, r, \gamma, \rho_0, \mathcal{G}, p_g, \phi \rangle$, where the reward is now goal-dependent, $r : \mathcal{S} \times \mathcal{A} \times \mathcal{G} \to \mathbb{R}$, and the objective is

$$J(\pi) = \mathbb{E}_{\substack{g \sim p_g \\ a_t \sim \pi(\cdot|s_t, g) \\ s_{t+1} \sim T(\cdot|s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, g) \right]$$

The agent must learn a *goal-conditioned policy* $\pi(a \mid s, g)$ that performs well across the entire distribution of goals, not just for one fixed task.

Remark. **Three notions of goal.** Liu et al. distinguish three concepts that are easily conflated:

- *Desired goal*: the target specified by the task or environment (e.g., "move the block to position x").
- *Achieved goal*: the goal corresponding to whatever state the agent actually reaches, $\phi(s_t)$.
- *Behavioral goal*: the goal the agent conditions on during a rollout, which may differ from the desired goal if sub-goal replacement is used.

The distinction between desired and achieved goals is central to hindsight methods (discussed below): when the agent fails to reach the desired goal, the achieved goal can still provide a useful learning signal.

When $\mathcal{G} \subseteq \mathcal{S}$. In the simplest and most common case, goals are states (or projections of states), so ϕ is either the identity or a coordinate projection. For example, in robotic manipulation, the full state might include joint angles, velocities, and the object pose, but the goal is only the target object position—a subspace of the state. This is the setting assumed by Contrastive RL, where goals $g \in \mathcal{S}$ are future states.

Relationship to multi-task learning. GCRL can be viewed as multi-task learning where each goal defines a separate task, and all tasks share the same dynamics but differ in their reward functions. The objective $J(\pi) = \mathbb{E}_{g \sim p_g}[J_g(\pi)]$ is an expectation over task-specific objectives, and the goal-conditioned policy shares parameters across all tasks. This is precisely the structure exploited by universal value function approximators. ■

Universal Value Function Approximators (Schaul et al., 2015)

The foundational architectural idea in GCRL is to parameterize value functions as functions of *both* the state and the goal. The **universal value function approximator** (UVFA) extends the standard value function to

$$V^\pi(s, g; \theta), \qquad Q^\pi(s, a, g; \theta)$$

where θ is shared across all goals. In practice, the goal g is simply concatenated with the state s (or with the state-action pair (s, a)) as input to the neural network. The UVFA can then be trained with any standard RL algorithm (DQN, SAC, etc.) by conditioning each update on the goal associated with that transition.

The key benefit is *generalization*: because the network must predict values for a continuum of goals, it is forced to learn features that capture the structure of the goal space. A UVFA trained on goals $g_1, \ldots, g_k$ can generalize to unseen goals g' if the representation is sufficiently smooth.

Remark. **Connection to Contrastive RL.** The bilinear form $\hat{C}_\theta(s, a, g) = \sigma(\phi_\theta(s, a)^\top \psi_\theta(g))$ Contrastive RL is a particular *factored* UVFA: rather than passing (s, a, g) through a monolithic network, it decomposes the value function into a state-action encoder and a goal encoder whose interaction is a dot product. This factorization is what enables the contrastive training procedure, but the underlying idea—a single network that takes goals as input—is exactly the UVFA framework.

Generalization vs. memorization. A UVFA that memorizes value tables for each goal separately would be no better than training independent agents. The power of the approach lies in the inductive bias of the shared network: goals that are "nearby" in some learned metric should have similar value landscapes. This is the multi-task generalization property that makes GCRL tractable even when the goal space is continuous and high-dimensional. ■

21.8.1 The Sparse Reward Problem and Hindsight Experience Replay

Sparse Rewards in GCRL

The most natural reward function for goal-reaching is a binary indicator:

$$r(s_t, a_t, g) = \mathbf{1}[\|\phi(s_{t+1}) - g\| \leqslant \epsilon]$$

for some tolerance $\epsilon > 0$. This reward is *maximally sparse*: the agent receives a signal of $+1$ only when it reaches the goal, and 0 everywhere else. In high-dimensional continuous control, reaching any specific goal by random exploration is astronomically unlikely, so the agent may collect thousands of episodes without ever observing a nonzero reward. Standard RL algorithms (DQN, SAC, policy gradients) all require informative reward signals to make progress, so they fail catastrophically under this level of sparsity.

An obvious fix is **reward shaping**: replace the sparse indicator with a dense distance-based reward, $\tilde{r}(s_t, a_t, g) = -\|\phi(s_{t+1}) - g\|$. While this provides gradient signal everywhere, it introduces a new problem: the shaped reward may create spurious local optima. For example, if the agent must move *away* from the goal before eventually reaching it (e.g., navigating

around an obstacle), the distance penalty punishes the correct initial behavior.

Remark. **Why sparsity is worse in GCRL than in standard RL.** In standard RL with a fixed task, the agent can occasionally stumble onto reward by chance if the reward region is not too small. In GCRL, the agent must reach a *specific* goal sampled from p_g, and the probability of randomly reaching any particular goal shrinks exponentially with the dimension of the goal space. Moreover, even when the agent does reach a goal, the experience is only directly useful for that one goal—it does not immediately help with other goals. This is the fundamental sample-efficiency bottleneck that all GCRL algorithms must address. ■

Hindsight Experience Replay (Andrychowicz et al., 2017)

Hindsight Experience Replay (HER) is the single most important algorithmic idea in GCRL. It is motivated by the observation that even a failed episode contains useful information: if the agent was trying to reach goal g but ended up at state s_T, then the trajectory is a *successful* demonstration of how to reach the achieved goal $\phi(s_T)$.

Formally, suppose the agent collects a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$ while pursuing desired goal g. HER stores τ in the replay buffer with the original goal, but also stores *additional copies* of τ with relabeled goals. Several relabeling strategies are possible:

- **Final**: relabel with the achieved goal at the end of the episode, $g' = \phi(s_T)$.
- **Future**: for each transition (s_t, a_t, s_{t+1}), relabel with a randomly chosen future achieved goal $g' = \phi(s_{t'})$ for $t' > t$.
- **Episode**: relabel with any achieved goal from the same episode.
- **Random**: relabel with an achieved goal from any episode in the buffer.

After relabeling, the reward is recomputed using the new goal: $r' = r(s_t, a_t, g')$. Since g' was actually reached by the trajectory, the relabeled transition has nonzero reward, transforming a failed experience into a successful one. The relabeled transitions are then used to train the UVFA via any off-policy algorithm (typically DDPG or SAC).

Remark. **Why HER works.** The key insight is that *off-policy algorithms do not care which goal the agent was pursuing when the data was collected*—they only care about the transition (s, a, s', r, g). By relabeling g, we create valid training data for a different goal without any additional environment interaction. In effect, every trajectory simultaneously teaches the agent how to reach many different goals, turning the sparsity problem on its head: even when the agent fails at the desired goal, it succeeds at many hindsight goals.

The "future" strategy dominates. Among the relabeling strategies, "future" tends to work best empirically because it creates a natural curriculum: for each transition (s_t, a_t), the relabeled goals are states that are actually reachable from s_t within the remaining horizon, providing appropriately challenging targets. The "final" strategy is a special case that only uses the terminal state, wasting the intermediate states.

Connection to Contrastive RL. There is a deep structural parallel between HER and the positive-pair construction in Contrastive RL. Both methods use *future states from the same trajectory* as tar-

gets: HER relabels goals with future achieved states, while Contrastive RL defines positive pairs as (s_t, s_{t+k}) for geometrically sampled k. In both cases, the fundamental mechanism is the same: repurposing the agent's own trajectory as a source of goal supervision. Contrastive RL can be seen as providing the theoretical justification for why this procedure works—it is implicitly performing TD learning on the discounted state occupancy.

Hindsight and inverse RL. Eysenbach et al. (2020) showed that HER can be interpreted as a special case of maximum-entropy inverse RL: relabeling goals is equivalent to inferring a reward function that makes the observed trajectory optimal, which is precisely the inverse RL problem. This connection suggests that more sophisticated inverse RL methods could yield improved relabeling strategies. ■

21.8.2 Goal Representations

Goal Representations Beyond State Vectors

The simplest GCRL setting defines $\mathcal{G} \subseteq \mathcal{S}$ with $\phi = \text{id}$, so that goals are target states. However, the goal-augmented MDP framework is flexible enough to accommodate richer representations:

- **Feature vectors**: the goal is a subspace of the state, e.g., the target position of an object while ignoring joint angles. Here ϕ is a coordinate projection.
- **Images**: the goal is a target image (e.g., a photo of the desired object arrangement). The mapping ϕ is typically a learned encoder (e.g., a VAE) that embeds both states and goals into a shared latent space where distances are meaningful.
- **Language**: the goal is a natural-language instruction (e.g., "go to the blue torch"). Here ϕ maps states to a language-compatible embedding, and the reward measures alignment between the achieved state embedding and the instruction embedding.
- **Expected returns**: the goal specifies a *desired cumulative reward* R, and the policy $\pi(a \mid s, R)$ is conditioned on the target return. This is the idea behind Decision Transformer (Chen et al., 2021) and upside-down RL (Schmidhuber, 2019), which treat reward-to-go as a goal and solve RL via supervised sequence modeling.

Remark. **Why the choice of goal representation matters.** The goal representation determines the structure of the reward function, the difficulty of generalization, and the applicable set of algorithms. For vector goals in a known subspace, rewards can be computed exactly via ℓ_2 distance and HER applies directly. For image goals, the reward computation requires a learned distance in embedding space, and the quality of the embedding critically affects learning. Language goals introduce an additional challenge: the mapping from language to grounded behavior requires compositional understanding.

Learned goal spaces. When raw goal representations are high-dimensional (e.g., images), it is often beneficial to learn a compact *latent goal space* $\mathcal{Z}$ via an encoder $E : \mathcal{G} \to \mathcal{Z}$. The policy and value function then operate on $\mathcal{Z}$ rather than the raw goal. This is precisely what methods like CIC do with their continuous skill space, and what variational goal-conditioned methods (e.g., RIG; Nair et al., 2018) do by training a VAE on goal images. The latent space should be structured so that Euclidean distance in $\mathcal{Z}$ corresponds to behavioral distance (i.e., how many

steps it takes to get from one goal to another), which connects directly to the quasimetric and contrastive representation ideas discussed elsewhere in this section. ■

21.8.3 Sub-Goal Generation and Curriculum Learning

Sub-Goal Decomposition

When the desired goal g is far from the agent's current capabilities (long-horizon tasks), even HER may not provide sufficient signal because the agent rarely visits states close to g. A natural solution, inspired by how humans decompose complex tasks, is to introduce *sub-goals*: intermediate waypoints $g_1, g_2, \ldots, g_n = g$ such that each consecutive sub-goal g_{i+1} is reachable from g_i with the agent's current skill level. The agent then conditions on each sub-goal in sequence. This decomposes the original long-horizon problem into a sequence of shorter, more tractable problems.

Formally, this replaces the environment's goal distribution p_g with a learned or heuristic distribution f over sub-goals:

$$J(\pi) = \mathbb{E}_{\substack{g \sim f \\ a_t \sim \pi(\cdot|s_t, g)}} \left[\sum_t \gamma^t r(s_t, a_t, g) \right]$$

The design of f is the central question, and several principles have been proposed.

Remark. **Intermediate difficulty.** A good sub-goal should be *just beyond* the agent's current abilities: too easy provides no new information, too hard reproduces the original sparse-reward problem. Florensa et al. (2018) operationalize this by training a GAN to generate goals of calibrated difficulty, where difficulty is measured by the agent's success rate. The generator learns to produce goals at the frontier of the agent's competence, creating an automatic curriculum.

Exploration-aware goal selection. Rather than targeting specific goals, another strategy maximizes the *coverage* of the goal space. Pitis et al. (2020) sample sub-goals that maximize the entropy of historically achieved goals, encouraging the agent to visit diverse states rather than repeatedly practicing easy goals. This is analogous to the entropy-based intrinsic rewards in CIC, but applied at the level of goal selection rather than reward design.

Graph search on the replay buffer. Eysenbach et al. (2019) propose a particularly elegant approach: build a graph whose nodes are states in the replay buffer and whose edge weights are distances estimated by the value function. Given a far-away target goal g, a shortest-path algorithm (e.g., Dijkstra) on this graph produces a sequence of waypoints through previously visited states, which the agent follows as a sequence of sub-goals. This uses the replay buffer not just as a data store but as a *map* of the environment.

Connection to hierarchical RL. Sub-goal generation is closely related to hierarchical RL, where a high-level policy selects sub-goals and a low-level policy executes them. The key difference in GCRL is that the same goal-conditioned policy serves as the low-level executor for all sub-goals, so the hierarchy is primarily a planning mechanism rather than a skill-learning mechanism. The high-level policy (or planner) proposes "where to go next," and the universal low-level policy handles "how to get there." ■

21.8.4 Synthesis: GCRL as a Unifying Lens

Remark. Goal-conditioned RL provides a unifying perspective on several themes developed throughout these notes:

From intrinsic motivation to goal-conditioning. The skill-discovery methods DIAYN and CIC train policies conditioned on a skill variable z, which is formally identical to a goal. The discriminator-based intrinsic reward $\log q_\phi(z \mid s)$ can be reinterpreted as a goal-conditioned reward that measures whether the agent has "reached" the behavior specified by z. GCRL makes this implicit goal structure explicit.

From contrastive representations to value functions. Contrastive RL (Eysenbach et al., 2022) showed that the GCRL value function has a contrastive factorization, so that learning a contrastive representation *is* learning a goal-conditioned value function. This resolves a longstanding question about why contrastive pre-training helps RL: it is not merely a useful initialization trick, but is performing implicit Bellman backup.

From HER to temporal contrastive learning. The "future" relabeling strategy in HER and the positive-pair construction in Contrastive RL are two implementations of the same principle: future states from the agent's own trajectory serve as natural goal supervision. HER operationalizes this via replay-buffer relabeling; Contrastive RL operationalizes it via the contrastive loss. The CTRL framework then extends this to multi-step compositional reasoning.

Remaining challenges. Despite the elegance of the framework, several open problems persist. Generalization across goal spaces remains difficult when goals are high-dimensional or semantically rich (images, language). The sparse reward problem, while ameliorated by HER, still limits performance in very long-horizon tasks where even hindsight relabeling cannot bridge the gap. And the interplay between goal-conditioned policies and exploration—how to efficiently discover which goals are *achievable*—remains an active area of research, connecting to the automatic curriculum and intrinsic motivation ideas discussed above. ■

21.9 Contrastive Representations for Temporal Reasoning

The contrastive RL framework above defines positives as *future states sampled at a geometrically distributed horizon*. A natural extension is to ask: can we encode more fine-grained temporal structure into the representation, for example to reason about multi-step relationships or to predict the sequence of states along a trajectory?

CTRL (Ziarko et al., 2022)

CTRL (*Contrastive Representations for Temporal Reasoning*) extends the contrastive RL framework to capture *multi-hop temporal relationships*. The goal is to learn representations $\phi : \mathcal{S} \to \mathbb{R}^d$ such that the inner product $\phi(s)^\top \phi(s')$ is informative about the *temporal distance* between states s and s' along the agent's trajectory.

Multi-step positive construction. Rather than sampling a single future state, CTRL defines an n-step positive pair (s_t, s_{t+n}) and a *compositional* positive (s_t, s_{t+n+m}) that can be decomposed via an intermediate state s_{t+n}:

$$\phi(s_t)^\top \phi(s_{t+n+m}) \approx \sum_u \phi(s_t)^\top \phi(u) \cdot \phi(u)^\top \phi(s_{t+n+m})$$

This *compositionality* property requires that the representation space supports a form of temporal chaining: representations should factor across time steps, enabling efficient multi-step planning.

The loss combines a standard InfoNCE contrastive term with a *temporal consistency regularizer* that enforces the compositional structure:

$$\mathcal{L}_{\text{CTRL}} = \mathcal{L}_{\text{NCE}}(\phi) + \lambda \mathcal{L}_{\text{comp}}(\phi)$$

where $\mathcal{L}_{\text{comp}}$ penalizes violations of the chaining identity above.

Remark. **Why compositionality matters for planning.** In model-based RL and hierarchical RL, the agent must reason about sequences of actions. A representation that satisfies the chaining identity allows the agent to estimate n-step reachability without rolling out n environment steps—it can combine k-step and $(n-k)$-step predictions algebraically. This is analogous to the way successor representations can be composed: if $M^\pi = (I - \gamma P^\pi)^{-1}$ is the discounted successor matrix, then multi-step predictions are encoded in M^π itself, and the contrastive representation is approximating this matrix in factored form.

Connection to Contrastive RL. The Eysenbach et al. framework from the previous section corresponds to the special case of CTRL where n is drawn from a geometric distribution and no compositionality constraint is imposed. CTRL can be seen as a structured extension that makes the temporal hierarchy explicit.

Practical benefits. Empirically, CTRL representations transfer more effectively across tasks that share temporal structure but differ in surface appearance. By encoding *how long it takes to get from s to s'* rather than just *whether s' is reachable from s*, the representation supports richer zero-shot generalization. ■

21.10 Summary and Connections

Remark. The following table situates the six methods covered in this section along two axes: *what the contrastive objective is applied to* (observations vs. state-goal pairs vs. temporal pairs) and *what it produces* (a better encoder, an intrinsic reward, or an implicit value function).

Method	Positive pairs	Output	Reward type
CURL	Augmented views of o_t	Encoder f_θ	Extrinsic
DIAYN	States + skill categories	Discriminator $q_\phi(z\|s)$	Intrinsic
MISL	Trajectory + skill	Trajectory discriminator	Intrinsic
CIC	States + continuous skills	Dual encoder (g_θ, h_ψ)	Intrinsic
Contrastive RL	(s, a) + future states	Value function C_θ	Goal-conditioned
CTRL	Multi-step state pairs	Compositional encoder	Goal / transfer

The unifying theme. All six methods train a representation by distinguishing *related pairs* (positive) from *unrelated pairs* (negative). What changes is the definition of relatedness: spatial proximity in observation space (CURL), behavioral identity under a skill (DIAYN, MISL, CIC), or temporal proximity in the agent's trajectory (Contrastive RL, CTRL). The deeper insight, formalized by Eysenbach et al., is that temporal relatedness is not merely a heuristic—it is the exact notion

of relatedness that makes contrastive learning equivalent to value function estimation. This gives the entire family a principled foundation: *contrastive RL is TD learning in representation space*. ■